

MadStree: tree formula

MadStree formula handbook

2026-08-21

Abstract

This handbook corresponds to **MadStree**; time-only sectors continue to be identified by fixed-length strings in the order of the root propagators. This document is the formula handbook of **MadStree**. The package targets arbitrary massive/massless mixed dS tree graphs. It does not generate a general system of IBP equations; instead, it assembles the master integrals, the iterative reduction and the block-triangular dlog-form differential equation directly from the topology, and a concrete usage guide will be added after the public interface stabilizes. This document rederives the required formulas from the tensor-product structure of the local function systems. The purely massive formulas of ref. [1] can be understood uniformly as Kronecker sums of several one-dimensional theta-free exponential systems and two-dimensional Hankel systems; each vertex carries exactly one such exponential parameter $k_{0,v}$, whose generator $ip_{0,v}$ is the tensor product of that one-dimensional factor with the identity on all the remaining factors. A massless full propagator containing theta cannot be identified with a single theta-free exponential factor: before reduction, it has $2 \otimes 2 = 4$ Hankel states at its two endpoints; after using the massless 00/11 and 10/01 relations, these four states are compressed by a constant projection into a two-dimensional state shared by the whole edge. This two-dimensional factor enters the time-IBPs of both endpoints, so a vertex-by-vertex tensor factorization fails, but the tensor-product/Kronecker-sum construction on the full graph still holds. All regular IBPs, iteration matrices and dlog diagonal blocks can therefore be written directly from the topology; theta derivatives only add fixed off-diagonal blocks connecting edge-contracted sectors, without first generating a large IBP system and then extracting the formulas from it. Any quantity quoted verbatim from [1] carries the superscript or subscript unreduced, whose meaning is given in Section 2: it is the formula of that paper, already rewritten in the $+\nu$, positive-prefactor convention of this document, but not yet combined with the extra massless reduction relations above.

Contents

1	Topology input convention	2
1.1	Loading and citation notice	3
2	Scope, reference conventions and conclusions	3
3	Local function systems: one-dimensional exponentials and two-dimensional h	5
3.1	One-dimensional systems of theta-free exponentials	5
3.2	The two-dimensional h system	5
3.3	Positive/negative prefactor conventions and the massless exponential kernel	6
4	The purely massive vertex formulas are Kronecker sums from the outset	6
5	From vertex spaces to the full-graph space	7
6	Massless full edges: from four states to a shared two-state	8
6.1	It is not a one-dimensional exponential factor	8
6.2	Two vertices act on the same two-dimensional factor	8
6.3	The contact selector is also obtained directly from the projection	9

7	Full-graph regular IBP with massless full edges	9
7.1	Positive/negative vertices, SK signs and Wick coordinates must be kept separate .	11
8	Direct iterative reduction: without first generating a large IBP system	12
8.1	The [1] binary order and the precise domain of the recurrences	12
8.2	Two-directional recurrences including the contact remaining term	13
9	The full-graph tensor formula for the dlog DE	17
9.1	Bit deletion, insertion and the massless quotient in Eq. (3.68) of [1]	18
10	The dimensionless h equation, physical quadratic-momentum terms and realification	20
11	Interface with the package conventions	21
11.1	The massless quotient basis of the package	21
11.2	The unique quotient and its four-state derivation space	22
11.3	$(-\tau)^A$ and normalized- J	22
11.4	The forward and inverse transformations of the H/h state vectors	23
11.5	Single-vertex families, local inverses and the complete reduction interface	24
11.6	Small-time endpoints and the safe time-IBP shift	25
12	Large pure-exponential-energy boundary and blow-up for arbitrary time-only graphs	26
12.1	Damping variables and auxiliary one-dimensional exponentials	26
12.2	Strict total time order, nested blow-up and theta fixing	26
12.3	Choosing the preferred rank from a user-preferred ordinary point	27
12.4	Contact subsectors inherit the same root rank	27
12.5	Controlled transport of an auxiliary external-leg energy to zero	29
12.6	The [2] infinity boundary and FlintNDE transport	30
12.7	Symbolic Laurent-support certificate and regulator error control	33
13	Typical usage examples	35
13.1	Single-vertex family: definition, reduction, and unified point input	35
13.2	Two-vertex massless Full edge: multiple sectors and points	35
13.3	Massive three-vertex tree: zero middle external-leg energy is not automatically singular	35
13.4	Three-vertex regulator: automatic Laurent support and incremental fitting	36
13.5	No physical external leg: private auxiliary energy to zero	36
14	Minimal algorithm and acceptance of the direct formula producer	36
15	Final assessment	37

1 Topology input convention

Every vertex explicitly sets its Schwinger–Keldysh contour branch through `vertexType -> "+"|"-`. A propagator no longer repeats `skType`, `sigma`, or endpoint signs; public `type` accepts only "massive" or "massless". A three-vertex chain is entered as

```
<|
"vertices" -> {
  <|"id"->1,"externalLegEnergy"->k1,"timePower"->a1,"vertexType"->"+"|>,
  <|"id"->2,"externalLegEnergy"->k2,"timePower"->a2,"vertexType"->"+"|>,
  <|"id"->3,"externalLegEnergy"->k3,"timePower"->a3,"vertexType"->"-"|>
```

```

},
"lines" -> {
  <|"type"->"massless", "endpoints"->{1,2}, "momentum"->q12|>,
  <|"type"->"massive", "endpoints"->{2,3}, "momentum"->q23, "nu"->nu23|>
}
|>

```

The ordered endpoint vertex types determine Full/Cross/External, the $++/--/+/-$ label, the Full sign, endpoint signs, and default Hankel branches. Names such as `masslessFull` and σ_e in formulas are derived internal physical data, not duplicated user fields. Vertex IDs may be integers. The public propagator interface neither defines nor reads an ID and are always numbered internally by their exact `lines` input positions; only an explicit `thetaBundles` declaration uses these position numbers.

The field `externalLegEnergy` is the theta-free external-leg exponent parameter attached to vertex v ; it does not mean that the vertex itself owns an energy. `vertexType=+` gives $\exp(-ik_v\tau_v)$ and `vertexType=-` gives $\exp(+ik_v\tau_v)$. The SK contour sign and this exponent sign are stored separately internally, but both are derived uniquely from the same `vertexType`. For a vertex without a physical external leg, `externalLegEnergy` may be omitted or set explicitly to zero. Both forms normalize to zero in the topology; the boundary producer uses a private auxiliary damping energy to construct the infinity boundary and transports that auxiliary coordinate back to the user-specified zero.

1.1 Loading and citation notice

The standard Wolfram entry point prints one citation notice per kernel only after every module has loaded successfully. The arXiv labels are clickable in a notebook, while a headless kernel keeps the complete URLs. The Python file protocol does not mix a text notice into JSON stdout; it stores the same MadStree citation list in response metadata, while direct adapter CLI use prints the complete text. When MadStree supports research, please cite:

1. Jiaqi Chen and Bo Feng, “Towards Systematic Evaluation of de Sitter Correlators via Generalized Integration-By-Parts Relations”, [arXiv:2401.00129](https://arxiv.org/abs/2401.00129).
2. Jiaqi Chen, Bo Feng and Yi-Xiao Tao, “Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations”, [arXiv:2411.03088](https://arxiv.org/abs/2411.03088).
3. MadStree package paper, arXiv identifier pending.

No arXiv identifier has yet been provided for the package paper, so this handbook does not invent a link or bibliographic metadata for it.

2 Scope, reference conventions and conclusions

This document takes as its formula sources Eqs. (3.1)–(3.5), (3.37), (3.47), (3.50), (3.54)–(3.55) and (3.64)–(3.68) of [1]. That reference uses τ^{ν_0} and

$$h(\nu, 0; z) = z^{-\nu} H_\nu(z), \quad h(\nu, 1; z) = \partial_z h(\nu, 0; z), \quad z = -k\tau, \quad (1)$$

as well as the natural binary basis. MadStree uses the input parameter ν as-is as the Hankel order: ν may be real, purely imaginary $i\mu$ (principal series/heavy fields), or complex, and no layer of the formulas and no part of the program ever takes an absolute value or a real part of ν . The prefactor convention is carried by a single sign s_ν ; the package uses only one function name h , and defines

$$\boxed{h(\nu, 0; z) = z^{s_\nu \nu} H_\nu(z)}, \quad s_\nu \in \{+1, -1\}. \quad (2)$$

The default `NuConvention` $\rightarrow$ "Positive" corresponds to $s_\nu = +1$; "Negative" corresponds to $s_\nu = -1$, which is the convention adopted by the printed formulas of refs. [1] and [2]. In both conventions the Hankel order is always the input ν itself, and the switch happens only in the powers of z . The two definitions do not differ merely by a constant; they differ by $z^{2\nu}$. Since the Hankel equation contains only ν^2 , the two sets of dimensionless equations of motion read respectively

$$h'' + \frac{1-2\nu}{z}h' + h = 0 \quad (s_\nu = +1), \quad h'' + \frac{1+2\nu}{z}h' + h = 0 \quad (s_\nu = -1). \quad (3)$$

Thus the two sets of algebraic formulas differ strictly only by $s_\nu \nu \rightarrow -s_\nu \nu$ (written below in the customary shorthand as $\nu \rightarrow -\nu$). This notation is only the algebraic consequence of flipping the sign of s_ν ; it does not replace the Hankel order by a conjugate order: what the integrand stores is always H_ν . One may also compare the positive-prefactor basis with the same function $h(-\nu, 0; z) = z^\nu H_{-\nu}(z)$ of the [1] convention taken at a negative parameter; $H_{-\nu}$ and H_ν are two independent solutions differing only by a kinematics-independent branch phase: the constant relation $H_{-\nu}^{(1)} = -e^{i\pi\nu} H_\nu^{(2)}$ absorbs this order difference entirely into the normalization, and for a purely imaginary order $\nu = i\mu$ it is exactly $e^{-\pi\mu} H_{-i\mu}^{(2)} = H_{i\mu}^{(2)}$ of Eq. (164) in Section 11. Let ν_{ref} denote the parameter of the same name in the printed formulas of that paper. It differs from the ν used by this document and by the default positive-prefactor implementation of MadStree only through

$$\boxed{\nu_{\text{ref}} = -\nu}. \quad (4)$$

When converting to the printed formulas of that paper through Eq. (4), this is not merely a replacement of the Hankel subscripts: every ν in the equations of motion, the Wronskian, M_1 , the contact powers, the recurrences and the dlog must be replaced simultaneously by ν_{ref} ; this replacement only rewrites the same integrand in the algebraic form of that paper's printed convention and does not change the Hankel order stored by the program. Equation (4) is therefore the only prefactor-convention conversion stated in this document: it has already been substituted into every formula quoted below, so from this section onwards the whole text is written in the $+\nu$, positive-prefactor convention, no formula appears in the other convention, and the reader never has to perform a further replacement. The base formula stored inside the program is exactly this $s_\nu = +1$, $+\nu$ form: everywhere an order appears in a formula, the program reads `formulaNu` := $s_\nu \nu$; with the default positive prefactor `formulaNu` = $+\nu$, and with "Negative" one has `formulaNu` = $-\nu = \nu_{\text{ref}}$, matching the printed formulas of that paper verbatim. In the package context `hankelOrder` always stores the input ν itself, while `formulaNu` and `hPrefactorPower` are identically equal to $s_\nu \nu$. The last section additionally gives the adaptation to the package's $(-\tau)^A$, normalized- J and sector normalization.

The core difference between this document and ref. [1] is the massless bulk propagator. The general iterative reduction Eqs. (3.37), (3.47), (3.54)–(3.55) and the subsector Eqs. (3.64)–(3.68) of that paper are all derived for a massive contracted edge, which carries an independent two-dimensional h state at each of its two endpoints. Eqs. (3.13)–(3.14) of that paper do note that the massless $G^{\pm\pm}$ also obeys vanishing and pinched relations, and its Section 3.6 gives the 0-fold vertex attached only to massless propagators, which has only $2^0 = 1$ master integral; that paper never uses this extra set of reduction relations to compress the four endpoint states of a massless full propagator into a single two-dimensional state shared by the whole edge. Therefore every quantity quoted verbatim from that paper carries a superscript or subscript unreduced in this document, to be read as “the formula of [1], already written in the $+\nu$ convention of this document via Eq. (4), but not yet specialized by the extra massless reduction relations of this document”. The 00/11 and 10/01 relations of Section 6 together with the $4 \rightarrow 2$ projection of Eq. (30) are precisely these extra relations; the final formulas of this document are the unreduced quantities plus these relations.

Two kinds of objects must first be distinguished:

- a pure exponential whose direction has already been chosen in an external leg or in G^{+-}/G^{-+} carries no theta and is only a one-dimensional function system;
- the massless full propagator in G^{++}/G^{--} contains two terms and a theta. Before reduction, like the massive full propagator, it has one two-dimensional h state at each endpoint, four states in total.

The core result of this document is that the massless extra relations compress the four states of the second kind into one shared two-dimensional factor, rather than mistreating it as two independent one-dimensional exponentials.

The conclusions at the formula level are organized in two tiers. The regular matrices of a fixed sector, the four-to-two-state projection and the common diagonalization are direct algebraic conclusions; the injection structure of contact into each lower sector can likewise be written directly, but the constant-residue dlog and flatness under arbitrary mixed/common-theta normalization still have to be verified independently. This document therefore does not automatically unlock the `PendingRederivation` of the existing package.

3 Local function systems: one-dimensional exponentials and two-dimensional h

3.1 One-dimensional systems of theta-free exponentials

Each vertex v carries exactly one theta-free exponential parameter, namely the input field `externalLegEnergy`, written $k_{0,v}$ in this document; its exponential factor is

$$E_v(\tau_v) = \exp(i\varsigma_v k_{0,v} \tau_v), \quad \varsigma_v = \pm 1, \quad (5)$$

where ς_v is derived uniquely from `vertexType`; see Section 1 and Eq. (53). The single user symbol $k_{0,v}$ takes as its value the sum of the momentum moduli of all theta-free external legs attached to v ; Eq. (3.1) of [1] likewise writes only one k_0 per vertex, which is the same summed quantity. From here on this document always uses the summed $k_{0,v}$, and introduces neither per-leg nor per-block exponential parameters nor any summation over them. Viewing E_v as a vector in the one-dimensional space $\mathcal{E}_v \simeq \mathbb{C}$, its time generator and signed energy are

$$Q_v = [i\varsigma_v k_{0,v}], \quad I_{\mathcal{E}_v} = [1], \quad p_{0,v} = \varsigma_v k_{0,v}, \quad \text{Emb}_v(Q_v) = ip_{0,v}I. \quad (6)$$

A one-dimensional exponential therefore does not increase the number of master integrals and does not create a slot: a vertex corresponds to a single one-dimensional factor $\mathcal{E}_v$, M_1 does not contain it, and M_0 contains only the scalar $ip_{0,v}I$. A massless cross/external branch also has no theta; it does not create a separate one-dimensional factor either, and its endpoint momentum moduli enter the same pure-exponential energy of the component according to Eq. (58). Every Hankel derivative term is also multiplied by the identity matrix of the one-dimensional factor, so all regular time derivatives remain Kronecker-sum terms of the same kind.

3.2 The two-dimensional h system

All formulas below are already written in the $+\nu$, positive-prefactor convention of this document. Let

$$\mathbf{h}(\nu; z) = \begin{pmatrix} h(\nu, 0; z) \\ h(\nu, 1; z) \end{pmatrix}, \quad P_1 = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}, \quad \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}. \quad (7)$$

Eqs. (3.2)–(3.4) of [1] are equivalent, in the convention of this document, to

$$\partial_\tau \mathbf{h} = \left[-ik\sigma_2 - \frac{1-2\nu}{\tau} P_1 \right] \mathbf{h}. \quad (8)$$

Here $-ik\sigma_2$ is a time-independent 2×2 matrix, and $-(1-2\nu)P_1/\tau$ merges with the lowering of the time power into M_1 .

The same system in the dimensionless variable $z = -k\tau$ satisfies

$$\partial_z^2 h(\nu, 0; z) + \frac{1-2\nu}{z} \partial_z h(\nu, 0; z) + h(\nu, 0; z) = 0. \quad (9)$$

The constant term here is 1, not k^2 . The k is recovered only through

$$\partial_\tau = -k\partial_z \quad (10)$$

This layering is the key to reusing the matrices of [1] later.

3.3 Positive/negative prefactor conventions and the massless exponential kernel

The Hankel equation depends only on the square of the order, but here the Hankel order is always the input ν as-is, and the two conventions only switch the powers on $z^{\pm\nu}$; Eq. (4) is the only conversion between the two prefactors and is not repeated below. In the default positive-prefactor convention of MadStree ($s_\nu = +1$, `formulaNu` = $+\nu$), when $\nu = 1/2$,

$$h(1/2, 0; z) = z^{1/2} H_{1/2}^{(1,2)}(z) \propto e^{\pm iz}, \quad (1-2\nu)|_{\nu=1/2} = 0. \quad (11)$$

Consequently

$$\partial_z^2 h(1/2, 0; z) + h(1/2, 0; z) = 0, \quad (12)$$

and the first derivative state is the branch eigenstate of the zeroth-order state,

$$h^{(1)}(1/2, 1) = +ih^{(1)}(1/2, 0), \quad h^{(2)}(1/2, 1) = -ih^{(2)}(1/2, 0). \quad (13)$$

The negative-prefactor option gives $z^{-1/2} H_{1/2}^{(1,2)} \propto z^{-1} e^{\pm iz}$, which is no longer a pure exponential; its formulas are obtained from the $+\nu$ base formulas by switching the order slot `formulaNu` from $+\nu$ to $-\nu$. This is only the algebraic consequence of flipping the sign of s_ν : the integrand still stores H_ν , and $H_{-\nu}$ differs from H_ν only by the constant $-e^{i\pi\nu}$ together with an interchange of branches. For a purely imaginary order $\nu = i\mu$ one especially cannot say that the order has been replaced by a conjugate order: $H_{-i\mu}$ and $H_{i\mu}$ are two independent solutions, differing only by the constant $-e^{-\pi\mu}$ together with an interchange of branches, i.e. by $e^{-\pi\mu} H_{-i\mu}^{(2)} = H_{i\mu}^{(2)}$ of Eq. (164). For a complex or purely imaginary order, a logarithm branch must be chosen first for $z^{\pm\nu}$; `Abs[nu]` or taking the real part of ν is never called anywhere.

4 The purely massive vertex formulas are Kronecker sums from the outset

Consider the p -fold vertex integral of [1]

$$V_v(\nu_{0,v}, \mathbf{a}) = \int d\tau_v \tau_v^{\nu_{0,v}} e^{ip_{0,v}\tau_v} \prod_{i=1}^p h(\nu_i, a_i; -k_i \tau_v), \quad a_i \in \{0, 1\}. \quad (14)$$

Its local space is

$$\mathcal{V}_v = \mathcal{E}_v \otimes \bigotimes_{i=1}^p \mathcal{H}_i, \quad \dim \mathcal{E}_v = 1, \quad \dim \mathcal{H}_i = 2. \quad (15)$$

For any local factor α , define the embedding

$$\text{Emb}_\alpha(A) = I_1 \otimes \cdots \otimes A_\alpha \otimes \cdots \otimes I_N. \quad (16)$$

Then a single time IBP of the integrand is performed: ∂_{τ_v} acts only on the factors belonging to vertex v —the one-dimensional exponential E_v of that vertex and the p massive endpoint h blocks attached to it—while the factors of the other vertices do not move and carry only the identity matrix; Eq. (3.37) of [1] is therefore written directly in the convention of this document as

$$M_{1,v}(\nu_{0,v}) = \nu_{0,v} I_v - \sum_{i=1}^p (1 - 2\nu_i) \text{Emb}_i(P_1), \quad (17)$$

$$M_{0,v} = ip_{0,v} I_v + \sum_{i=1}^p \text{Emb}_i(-ik_i \sigma_2). \quad (18)$$

Using $P_1 = (I - \sigma_3)/2$, the first equation becomes the Λ_3 form of that paper, written in the convention of this document,

$$M_{1,v} = \sum_i \left(\frac{1}{2} - \nu_i \right) \Lambda_3^{(i)} + \left(\nu_{0,v} - \frac{p}{2} + \sum_i \nu_i \right) I_v. \quad (19)$$

Thus $p_0 I$ and $k_i \Lambda_2^{(i)}$ are not essentially different: the former is the generator of the one-dimensional exponential factor of that vertex, the latter comes from the two-dimensional h factors; both are local generators multiplied by all the spectator identities.

Let $\mathbf{f}_v(a)$ denote the 2^p -dimensional integral vector obtained after shifting $\nu_{0,v}$ by the integer a . Ignoring the theta remaining term,

$$M_{1,v}(\nu_{0,v} + a) \mathbf{f}_v(a - 1) + M_{0,v} \mathbf{f}_v(a) = 0. \quad (20)$$

One lowering step is directly

$$A_{-,v}(a) = -M_{1,v}(\nu_{0,v} + a)^{-1} M_{0,v}. \quad (21)$$

The constant matrix of [1]

$$T = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -i \\ -i & 1 \end{pmatrix}, \quad T \sigma_2 T^{-1} = \sigma_3, \quad (22)$$

simultaneously diagonalizes all two-dimensional local generators in M_0 . Let $T_v = T^{\otimes p}$; then

$$\widetilde{M}_{0,v} = T_v M_{0,v} T_v^{-1} \quad (23)$$

is diagonal, and one raising step is

$$A_{+,v}(a) = -T_v^{-1} \widetilde{M}_{0,v}^{-1} T_v M_{1,v}(\nu_{0,v} + a + 1). \quad (24)$$

So the purely massive formulas never need a general large-matrix inversion from the outset: M_1 is diagonal in the original basis, and $\widetilde{M}_0$ is diagonal in the T basis.

5 From vertex spaces to the full-graph space

Let V_s be the set of active vertices of a fixed sector s . Before the massless extra relations are used, the h state at each propagator endpoint is an independent two-dimensional factor, and each active vertex carries one one-dimensional exponential factor. The regular full-graph space can therefore be written as

$$\mathcal{V}_s^{\text{unreduced}} = \bigotimes_{v \in V_s} \mathcal{E}_v \otimes \bigotimes_{\text{all active endpoints } i} \mathcal{H}_i. \quad (25)$$

A one-dimensional factor does not create a slot and contributes only the scalar $p_{0,v}$. The $M_{0,v}$ and $M_{1,v}$ of every vertex are embedded into this common space by Eq. (16). In the time IBP with

respect to τ_v , ∂_{τ_v} acts only on the building blocks belonging to that vertex: the one-dimensional exponential factor of the vertex, the massive endpoint h blocks attached to it, and, for theta-free cross propagators, the pure exponential branch in which it participates as an endpoint; the factors of the other vertices carry only the identity matrix. For a purely massive graph, or for theta-free cross propagators, the regular operator of vertex v is therefore supported on its own local factors, so the vertex families of different vertices factorize independently. The theta derivatives of G^{++}/G^{--} do not belong to the regular operator; they are added separately as lower-sector remaining terms.

This full-graph formulation is more fundamental than writing the formula once for each vertex. The latter is valid only because, in the purely massive case, the regular operators of different vertices are supported on mutually disjoint two-dimensional factors.

6 Massless full edges: from four states to a shared two-state

6.1 It is not a one-dimensional exponential factor

Consider a massless full edge $e = (u_e, v_e)$ containing theta. Before the extra relations are used, it is exactly like a massive propagator: endpoint u_e has two states $a \in \{0, 1\}$, endpoint v_e has two states $b \in \{0, 1\}$, so the local space of the edge is

$$\mathcal{H}_{e,u} \otimes \mathcal{H}_{e,v} \simeq \mathbb{C}^4. \quad (26)$$

In the natural order of ref. [1] take

$$\mathbf{F}_e^{(4)} = (F_{00}, F_{01}, F_{10}, F_{11})^T. \quad (27)$$

By Eq. (13), in the dimensionless h-state convention of ref. [1],

$$F_{01} = -F_{10}, \quad F_{11} = F_{00}. \quad (28)$$

No k^2 is missing here: each F_{ab} uses $h_1 = -k^{-1}\partial_\tau h_0$, not the physical endpoint derivative from which k has not been removed. It is also not the algebraic quotient of the package, which factors out the phase of each endpoint derivative; the 11 relation of the latter carries a minus sign. The physical relation is restored in Sec. 10, and the package quotient is given in Sec. 11.

Choose the shared two-state

$$\mathbf{g}_e = (F_{00}, F_{10})^T \in \mathcal{M}_e \simeq \mathbb{C}^2. \quad (29)$$

The constant embedding and projection between the four states and the two states are

$$\boxed{\mathbf{F}_e^{(4)} = S_e \mathbf{g}_e, \quad S_e = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad P_e = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad P_e S_e = I_2.} \quad (30)$$

Any original four-dimensional operator that preserves the relation subspace can be reduced directly to

$$O_e^{\text{red}} = P_e O_e S_e. \quad (31)$$

6.2 Two vertices act on the same two-dimensional factor

Before reduction, the constant h generator of endpoint u_e is $-iq_e \sigma_2 \otimes I_2$, and that of endpoint v_e is $-iq_e I_2 \otimes \sigma_2$. Applying Eq. (30) directly gives

$$P_e(\sigma_2 \otimes I_2)S_e = \sigma_2, \quad (32)$$

$$P_e(I_2 \otimes \sigma_2)S_e = -\sigma_2. \quad (33)$$

Therefore, once the four states are reduced to two, the two endpoints no longer each have an independent tensor factor; instead they act with $+\sigma_2$ and $-\sigma_2$ respectively on the same $\mathcal{M}_e$. This is precisely the only regular reason for the failure of vertex-by-vertex factorization.

Importantly, the full-graph tensor-product structure does not fail. What fails is the decomposition

$$\mathcal{V}_s \stackrel{?}{=} \bigotimes_{v \in V_s} \mathcal{V}_v, \quad (34)$$

because the shared $\mathcal{M}_e$ cannot belong independently to both u_e and v_e ; but all vertex operators are still Kronecker sums on the same full-graph space.

6.3 The contact selector is also obtained directly from the projection

Separating the overall scalar from the four-state remaining-term pattern of Eq. (3.65) of [1], the pattern is

$$\mathbf{r}_e^{(4)} = (0, -1, +1, 0)^T. \quad (35)$$

It satisfies exactly

$$\mathbf{r}_e^{(4)} = S_e |1\rangle_e, \quad |1\rangle_e = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (36)$$

So contact does not need to be re-solved: in the shared two-state it is simply the fixed column vector $|1\rangle_e$. The overall scalar $w_e^{(v)}$ depends on the two-state basis. Let $\sigma_e = +1, -1$ denote G^{++}, G^{--} respectively; in the h-state basis of ref. [1] used in this section, the ordered first and second endpoints are

$$w_{e,h}^{(u_e)} = +2i\sigma_e, \quad w_{e,h}^{(v_e)} = -2i\sigma_e. \quad (37)$$

The constant basis change of Sec. 11 converts these exactly into the package's $-2, +2$. One only needs, in addition, to multiply by the common-theta odd-subset coefficient and the sector-normalization ratio; a massless edge carries no extra Hankel Wronskian prefactor.

7 Full-graph regular IBP with massless full edges

Fix a contact-reachable sector s and let V_s be its set of active vertices. Write

- $H(s)$ for all still-active massive endpoint h slots;
- $E_m(s)$ for all uncontracted massless full edges;

The reduced full-graph space is

$$\mathcal{V}_s = \bigotimes_{v \in V_s} \mathcal{E}_v \otimes \bigotimes_{i \in H(s)} \mathcal{H}_i \otimes \bigotimes_{e \in E_m(s)} \mathcal{M}_e. \quad (38)$$

Since $\dim \mathcal{E}_v = 1$, its dimension is

$$\dim \mathcal{V}_s = 2^{|H(s)| + |E_m(s)|}. \quad (39)$$

This gives a slot registry that can be initialized directly. Each slot records a stable number, its type, the root line/vertex, the user momentum name, and the basis and SK/Hankel metadata; each root vertex v additionally stores its incidence list

$$\mathcal{B}(v) = \{i \in H : v(i) = v\} \cup \{(e, \epsilon_{ve}) : e \in E_m, v \in e\}. \quad (40)$$

One-dimensional exponentials do not enter this registry: they occur in M_0 only as the scalars $p_{0,v}$, whereas the user still supplies numerical points and differentiation variables in terms of their own $k_{0,v}$ and the propagator moduli q_e ; the two coordinate sets are related by the pullback of Eq. (57).

The same label e of a massless full edge appears in the lists of both vertices; this is exactly the data expression of “shared, not copied.”

For each ordered edge $e = (u_e, v_e)$ define the incidence

$$\epsilon_{ve} = \begin{cases} +1, & v = u_e, \\ -1, & v = v_e, \\ 0, & v \notin e. \end{cases} \quad (41)$$

In the h-state basis of ref. [1], the choice of Hankel branch changes the particular solution but not the matrix $-iq_e\sigma_2$ of the first-order system. Therefore the branch sign of G^{++}/G^{--} is not written into the generator here; that sign appears explicitly when changing to the package quotient basis.

The contribution of each kind of atom to a vertex v can be written uniformly as

$$\Delta M_{1;v}^{(E_v)} = 0, \quad \Delta M_{0;v}^{(E_v)} = ip_{0,v}I, \quad v \in V_s, \quad (42)$$

$$\Delta M_{1;v}^{(h_i)} = -(1 - 2\nu_i)P_1, \quad \Delta M_{0;v}^{(h_i)} = -ik_i\sigma_2, \quad i \in H_v, \quad (43)$$

$$\Delta M_{1;u_e}^{(m_e)} = \Delta M_{1;v_e}^{(m_e)} = 0, \quad \Delta M_{0;u_e}^{(m_e)} = -iq_e\sigma_2, \quad \Delta M_{0;v_e}^{(m_e)} = +iq_e\sigma_2. \quad (44)$$

The ν_i in Eq. (43) are already the parameters of the $+\nu$ convention of this document; the base formula stored inside the program is exactly this $s_\nu = +1, +\nu$ form, where the order slot reads `formulaNu` = $s_\nu\nu_i$, i.e. $+\nu_i$ under the default positive prefactor, so the $(1 - 2\text{formulaNu})$ of the code agrees directly with the $(1 - 2\nu_i)$ of this document. When comparing verbatim with the printed formulas of that paper, replace the order by $\nu_{\text{ref}} = -\nu_i$ through Eq. (4); the $(1 - 2\nu_i)$ and $(2\nu_i - 1)$ of this document then reproduce the printed $(2\nu_{\text{ref}} + 1)$ and $(-2\nu_{\text{ref}} - 1)$ of that paper. The global M_1 also receives the component time power $\nu_{0,v}I$, or the package’s A_vI . The two endpoints of Eq. (44) act on the same two-dimensional slot and cannot each multiply out a new slot.

On the full-graph space (38), the regular matrices of each vertex are directly

$$M_{1;v,s}(\nu_{0,v}) = \nu_{0,v}I_s - \sum_{i \in H_v(s)} (1 - 2\nu_i) \text{Emb}_i(P_1), \quad (45)$$

$$M_{0;v,s} = ip_{0,v}I_s + \sum_{i \in H_v(s)} \text{Emb}_i(-ik_i\sigma_2) + \sum_{e \in E_m(s)} \text{Emb}_e(-i\epsilon_{ve}q_e\sigma_2). \quad (46)$$

A massless edge does not enter M_1 : the default $\nu = 1/2$ makes the $1 - 2\nu$ of Eq. (11) vanish identically. The same $\text{Emb}_e(\sigma_2)$ appears in both $M_{0;u_e,s}$ and $M_{0;v_e,s}$ at the two ends of the edge, with opposite incidence signs.

All $M_{0;v,s}$ are made of σ_2 on different slots, or of the same σ_2 on the same shared slot, so they commute with one another and can be simultaneously diagonalized by a single global constant transformation

$$U_s = \bigotimes_{i \in H(s)} T_i \otimes \bigotimes_{e \in E_m(s)} T_e \quad (47)$$

simultaneously; the one-dimensional exponential factors only contribute [1] and are not written explicitly.

This step does not mean that arbitrary Pauli combinations can be handled this way. For any two-dimensional slot α , write the local part of each vertex on that slot as

$$Q_{v\alpha} = c_{0,v\alpha}I + \mathbf{c}_{v\alpha} \cdot \boldsymbol{\sigma}. \quad (48)$$

The condition for a common diagonalizer to exist slot by slot is

$$[Q_{v\alpha}, Q_{w\alpha}] = 0 \quad \Longleftrightarrow \quad \mathbf{c}_{v\alpha} \times \mathbf{c}_{w\alpha} = 0 \quad (\text{all } v, w). \quad (49)$$

That is, all Pauli directions on the same slot must be collinear. In the h-state of ref. [1] at present, only σ_2 appears on every massive/shared slot; in the package quotient, massive slots carry only σ_2 and each massless shared slot carries only σ_1 , so the global diagonalizer is changed to

$$U_s^J = \bigotimes_{i \in H(s)} T_i \otimes \bigotimes_{e \in E_m(s)} H_e, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \quad (50)$$

If, in the future, a shared slot simultaneously receives non-collinear σ_1 and σ_2 , a single 2×2 matrix might still diagonalize it separately, but different vertices would no longer share a common U_s ; in that case one must not continue replacing the large-matrix inversion with a uniform letter-by-letter reciprocal.

Let the massive bits be $a_i \in \{0, 1\}$ and the shared massless-edge bits be $r_e \in \{0, 1\}$. Then

$$U_s M_{0;v,s} U_s^{-1} = \text{diag}(i\kappa_{v,s;\mathbf{a},\mathbf{r}}), \quad (51)$$

where

$$\boxed{\kappa_{v,s;\mathbf{a},\mathbf{r}} = p_{0,v} + \sum_{i \in H_v(s)} (2a_i - 1)k_i + \sum_{e \in E_m(s)} \epsilon_{ve}(2r_e - 1)q_e.} \quad (52)$$

The same r_e enters the letters of both u_e and v_e with opposite signs. This is the complete regular content of the shared-edge correlation.

The massless quotient therefore does not prevent the use of Eqs. (45)–(46) for a single vertex: given v , one only needs to add the terms block by block from $\mathcal{B}(v)$. The meaning of this vertex-by-vertex assembly is stated here explicitly: in the time IBP with respect to τ_v , ∂_{τ_v} acts only on the building blocks belonging to that vertex—the one-dimensional exponential factor of the vertex, the massive endpoint h blocks attached to it, and the massless shared blocks in which it participates as an endpoint (with incidence $\epsilon_{ve} = \pm 1$); the factors of the remaining vertices carry only the identity matrix, so Eqs. (45) and (46) are exactly the Kronecker sum of that vertex's local atoms. What changes is not this vertex-by-vertex assembly rule but the support relation between the matrices of different vertices: $M_{0;u_e,s}$ and $M_{0;v_e,s}$ contain opposite coefficients on the same slot e , rather than acting on two unrelated slots. The two vertex-by-vertex recurrences share a state bit, yet each is still a closed-form matrix formula; there is no need to assemble a large system of IBP equations first.

7.1 Positive/negative vertices, SK signs and Wick coordinates must be kept separate

The theta-free exponential of Eq. (5) is unique per vertex and reads

$$\exp(i\varsigma_v k_{0,v} \tau_v), \quad \varsigma_v = +1 \text{ or } -1, \quad (53)$$

where ς_v is derived uniquely from **vertexType**: "+" gives $\varsigma_v = -1$ and "-" gives $\varsigma_v = +1$. ς_v is the vertex phase sign; it is neither the G^{++}/G^{--} label σ_e of a massless propagator nor the ordered-endpoint incidence ϵ_{ve} .

The only effect of positive and negative vertices on the regular formulas is

$$p_{0,v} = \varsigma_v k_{0,v}, \quad M_{0;v,s} \supset i p_{0,v} I_s. \quad (54)$$

M_1 does not contain the one-dimensional exponential; the P_1, σ_2 atoms of the massive h are likewise unchanged by the Hankel branch. The difference between G^{++} and G^{--} is instead stored in the theta branch, the Wronskian/contact weights, and in D_{σ_e} of the package quotient; it cannot be replaced by ς_v .

Now set $\tau_v = -t_v < 0$. Along the damping rays of the two kinds of vertices define uniformly

$$K_v = i\varsigma_v k_{0,v} > 0, \quad k_{0,v} = -i\varsigma_v K_v. \quad (55)$$

Then

$$\exp(i\varsigma_v k_{0,v} \tau_v) = \exp(-K_v t_v). \quad (56)$$

Therefore, for $\varsigma_v = +1$ one may write $k_{0,v} \rightarrow -i \text{pik}_{0,v}$ in the reference notation, while for $\varsigma_v = -1$ one writes $k_{0,v} \rightarrow +i \text{mik}_{0,v}$; in both cases $\text{pik}_{0,v}$ and $\text{mik}_{0,v}$ are positive. The main text uses these names only for communication with the literature; internally, the program should generate the stable K_v coordinates at initialization.

The user still supplies points with their own $k_{0,v}$ and all propagator moduli q_e from initialization. When the connection is output in user coordinates, only the invertible pullback

$$dp_{0,v} = \varsigma_v dk_{0,v}, \quad dK_v = i\varsigma_v dk_{0,v}; \quad (57)$$

is performed; the user's q_e are not silently renamed **pik** or **mik**. When contact merges a set of root vertices into a component C , the pure-exponential energy and the damping variable of that component are

$$p_{0,C} = \sum_{v \in C} \varsigma_v k_{0,v} + \sum_{\substack{e \text{ massless cross/external} \\ v \in e \cap C}} (-\varsigma_v) q_e, \quad K_C = \sum_{v \in C} K_v. \quad (58)$$

The second term comes from the theta-free massless cross/external branches: such a branch creates no slot, and only adds the momentum moduli of those of its endpoints that lie in C , each with the SK contour sign $-\varsigma_v$ of that endpoint, into the same scalar; the sum runs over active lines only. For a single-vertex component, Eq. (58) reduces to the $p_{0,v}$ of Eq. (54).

8 Direct iterative reduction: without first generating a large IBP system

8.1 The [1] binary order and the precise domain of the recurrences

Fix a sector s , let n_s^{slot} be the number of two-state factors in the sector and c_s the number of time components after contact. Following the binary representation of [1], write a state as

$$\mathbf{a} = (a_1, \dots, a_{n_s^{\text{slot}}}), \quad a_i \in \{0, 1\}. \quad (59)$$

The state order is fixed to

$$\mathbf{a}(m) = \text{IntegerDigits}(m, 2, n_s^{\text{slot}}), \quad 0 \leq m < 2^{n_s^{\text{slot}}}. \quad (60)$$

The first bit is the most significant; in a Kronecker product the left slots vary slowly and the right slots vary quickly. For any bit string, $\text{Del}_i \mathbf{a}$ deletes the i -th bit and $\text{Ins}_i(\hat{\mathbf{c}}, x)$ inserts $x \in \{0, 1\}$ at position i . These conventions are exactly in the same order as `IntegerDigits[... , 2, n]`, `Delete` and `Insert` of the reference code.

A native integral of MadStree can be written as

$$J_s(\mathbf{n}; \mathbf{a}) \equiv \text{MSIntegral}[s, \mathbf{n}, \mathbf{a}], \quad \mathbf{n} \in \mathbb{Z}^{c_s}, \quad \mathbf{a} \in \{0, 1\}^{n_s^{\text{slot}}}. \quad (61)$$

Here s is not a label recovered from the local $\mathbf{n}, \mathbf{a}$. Let the root propagators be, in the initialization order, $E = (e_1, \dots, e_{N_E})$, and let the contracted canonical set be L_s^{ctr} . Define the fixed-length string

$$\text{key}(s) = q_1 q_2 \cdots q_{N_E}, \quad q_j = \begin{cases} 0, & e_j \in L_s^{\text{ctr}}, \\ 1, & e_j \notin L_s^{\text{ctr}}. \end{cases} \quad (62)$$

Propagators that cannot undergo contact shrink always have $q_j = 1$, and the top sector is the all-1 string. The key is a string, not a binary integer; leading zeros cannot be dropped. For example, if $E = (e_1, e_2, e_3, e_4)$ and $L_s^{\text{ctr}} = \{e_1, e_2, e_4\}$, the key is "0010". The full integral head always puts

this key in the first argument. Hence two subsectors remain different `MSIntegral` expressions even if $c_s, n_s^{\text{slot}}, \mathbf{n}, \mathbf{a}$ are all identical. If the same contraction set is reached through different contact paths, it is merged into the same sector. Before returning the context, `MSInitTree` generates a `sectorIdentityCertificate` that checks in one pass the canonical contraction sets, the sector keys, the full master expressions, the consecutive global indices, and the SHA-256 digest of the complete ordered master table; any collision returns `SectorIdentityCollision`, so that no ambiguity is left to the recurrence or DE consumers.

Thus the input domain of `MSReduce` is not “arbitrary dS integrals” but the space spanned, in a fixed initialization context, by the following legal objects

$$\boxed{\text{span}\left\{J_s(\mathbf{n}; \mathbf{a}) \mid s \in \text{ContactDAG}, \mathbf{n} \in \mathbb{Z}^{c_s}, \mathbf{a} \in \{0, 1\}^{n_s^{\text{slot}}}\right\}}. \quad (63)$$

That is, one may choose arbitrary integer time shifts and all binary states in each contact-reachable sector, and take finite linear combinations of such legal objects; this set of recurrences cannot introduce new propagator powers, new discrete indices, sectors outside the context, or another function family.

8.2 Two-directional recurrences including the contact remaining term

Let $\mathbf{F}_s(\mathbf{n})$ denote the ordered master vector of sector s , where n_v shifts the local time power $\nu_{0,v}$ of vertex v itself—the v -th slot of the package’s `timeShifts` (equivalently, the order parameter of the diagonal raising operator A_v). The theta remaining term is written as a linear combination of lower-sector vectors

$$\mathcal{R}_{v,s}(\mathbf{n}) = \sum_{t \prec s} C_{st}^{(v)}(\mathbf{n}) \mathbf{F}_t(\mathbf{n}_t). \quad (64)$$

More concretely, for an ordinary one-edge contact it can be written as

$$\mathcal{R}_{v,s}(\mathbf{n}) = \sum_{\substack{e \in E_{\text{full}}(s) \\ v \in e}} C_{s,s/e}^{(v,e)} \mathbf{F}_{s/e}(\chi_e(\mathbf{n})). \quad (65)$$

χ_e merges the time shifts belonging to the two ends of the edge and includes the fixed shift by which the delta contact makes the child base power one less. This parent-to-child structure is identical for massive and massless edges; only the local matrices $C^{(v,e)}$ differ. A massive edge uses the complementary endpoint bits and signs of Eq. (3.65) of [1]; a massless edge first takes the $4 \rightarrow 2$ quotient and then uses the shared-bit selector of Eq. (78) together with the $-2, +2$ of Eq. (119). Hence massless contraction does not need a second set of recurrence formulas.

The package fixes the child base power deliberately according to the delta contact: $\mathcal{R}(\mathbf{0})$ lands on child shift -1 , while the $\mathcal{R}(e_v)$ needed by the DE lands exactly on child shift 0 . Below, the remaining term used by the raising step is denoted $R^{(1)} := \mathcal{R}_{v,s}(e_v)$ and the one used by the lowering step $\mathcal{R}_{v,s}(\mathbf{0})$ is denoted $R^{(0)}$; they are the two remainders of the same one-step relation evaluated at different indices, their targets differ by one child shift, and they are not interchangeable. Hence $R^{(1)}$ of an ordinary one-edge massive/massless contact already hits the child master directly; there is no problem of “a dlog master being chosen but the iterative reduction failing to reach it.”

The complete one-step IBP at vertex v is

$$M_{1;v,s}(\nu_{0,v} + n_v) \mathbf{F}_s(\mathbf{n} - \mathbf{e}_v) + M_{0;v,s} \mathbf{F}_s(\mathbf{n}) + \mathcal{R}_{v,s}(\mathbf{n}) = 0. \quad (66)$$

Eq. (66) is written vertex by vertex: each vertex v has its own one-step relation, and that relation is exactly what the time IBP ∂_{τ_v} of that vertex produces. ∂_{τ_v} acts only on the building blocks belonging to vertex v —the one-dimensional exponential factor of the vertex, the massive endpoint blocks attached to it, and the massless shared blocks in which it participates as an endpoint—while the factors of the other vertices carry only the identity matrix. Therefore $\nu_{0,v}, M_{1;v,s}, M_{0;v,s},$

$\mathcal{R}_{v,s}$ and the later $A_{\pm,v,s}$ all read only the local atoms of vertex v , and the shift directions $\mathbf{e}_v$ do not mix between different vertices: the iterative reduction of the whole family proceeds one vertex at a time, and **MSReduce** likewise applies the relation of each component separately.

One lowering step is therefore directly

$$\mathbf{F}_s(\mathbf{n} - \mathbf{e}_v) = A_{-,v,s}(\mathbf{n})\mathbf{F}_s(\mathbf{n}) - M_{1;v,s}^{-1}\mathcal{R}_{v,s}(\mathbf{n}), \quad (67)$$

$$A_{-,v,s}(\mathbf{n}) = -M_{1;v,s}(\nu_{0,v} + n_v)^{-1}M_{0;v,s}. \quad (68)$$

One raising step uses the diagonal matrix

$$\widetilde{M}_{0;v,s} = U_s M_{0;v,s} U_s^{-1} \quad (69)$$

and gives

$$\mathbf{F}_s(\mathbf{n} + \mathbf{e}_v) = A_{+,v,s}(\mathbf{n})\mathbf{F}_s(\mathbf{n}) - U_s^{-1}\widetilde{M}_{0;v,s}^{-1}U_s\mathcal{R}_{v,s}(\mathbf{n} + \mathbf{e}_v), \quad (70)$$

$$A_{+,v,s}(\mathbf{n}) = -U_s^{-1}\widetilde{M}_{0;v,s}^{-1}U_s M_{1;v,s}(\nu_{0,v} + n_v + 1). \quad (71)$$

These are the direct generalizations of Eqs. (3.37), (3.47), (3.50) and (3.66) of ref. [1] to the shared massless-edge space. What still needs to be inverted is only the diagonal M_1 in the original basis and the diagonal $\widetilde{M}_0$ in the U_s basis. The complete singular layers at general indices can now be obtained directly.

Before explaining the singular layers, one must distinguish the h block in the formulas from the complete physical mode. If d is the spatial dimension and contributions such as the curvature coupling are absorbed into m_{eff} , the scalar index satisfies

$$\nu^2 = \frac{d^2}{4} - \frac{m_{\text{eff}}^2}{H^2}. \quad (72)$$

Four-dimensional de Sitter has $d = 3$, and the complete mode contains

$$u_k(\tau) \propto (-\tau)^{3/2} H_\nu^{(1,2)}(-k\tau), \quad (73)$$

rather than the Hankel function alone. For general d the two late-time branches are

$$u_k(\tau) \sim c_-(-\tau)^{d/2-\nu} + c_+(-\tau)^{d/2+\nu}. \quad (74)$$

For a minimally coupled massless scalar, $\nu = d/2$, and the leading branch is exactly the zeroth power while the other branch is the d -th power. For a light field with $0 < m_{\text{eff}}^2 < d^2 H^2/4$, one has $0 < \nu < d/2$ and both branches have positive real part; for a heavy field $\nu = i\mu$, both real parts equal $d/2 > 0$. Hence the mass term indeed enters Eq. (72) with a minus sign, and the natural late-time powers of a generic massive mode do not by themselves produce boundary divergences.

$h = z^{\pm\nu} H_\nu$ only redistributes the $\pm\nu$ power of Eq. (74) between the explicit $(-\tau)^A$ and the interior of the block; the same convention must be fixed after initialization, but the complete physical power is unchanged. If the master integrals keep the natural $(-\tau)^{d/2}$ prefactor of each propagator, both massive branches decay at $\tau \rightarrow 0$. Although the massless leading branch is a constant, a total-derivative primitive used by the IBP still tends to zero as long as it carries one more positive explicit $(-\tau)$ power; what is genuinely dangerous is the total primitive power dropping to zero or below. This corresponds exactly to the layers below where the M_1 eigenvalue vanishes or is crossed; one must not add extra boundary terms merely because “the massless mode is itself a constant,” nor continue using the boundary-discarding recurrence on the zero layer.

For a state $(\mathbf{a}, \mathbf{r})$, the M_1 eigenvalue is

$$\lambda_{1;v,\mathbf{a}}(n_v) = \nu_{0,v} + n_v - \sum_{i \in H_v} (1 - 2\nu_i) a_i, \quad (75)$$

which is independent of the massless shared bits $\mathbf{r}$ and of all kinematics. When lowering $n_v \rightarrow n_v - 1$, only

$$\lambda_{1;v,\mathbf{a}}(n_v) = 0 \quad (76)$$

produces an M_1^{-1} pole. A general mixed/Hankel family still needs to be judged against the concrete small- τ asymptotics; but a purely massless vertex has no massive endpoint terms, and setting $\alpha_v = \nu_{0,v} + n_v$ gives $M_{1;v} = \alpha_v I$, while the corresponding endpoint integral indeed contains $\int_0^\infty dt_v t_v^{\alpha_v-1} e^{-\kappa t_v} = \Gamma(\alpha_v) \kappa^{-\alpha_v}$. Hence when the whole M_1 vanishes at $\alpha_v = 0$, the unregularized Gamma integral itself diverges and the boundary-discarding IBP cannot be used directly. One should first take $\alpha_v \rightarrow \alpha_v + \delta$ as analytic regularization; in the suitably regularized generic- δ system M_1 is invertible, and after the recurrences are completed one expands and takes $\delta \rightarrow 0$, so no physical failure of the recurrences appears. For massive states, if one instead uses the natural time powers without the absorbed $H \rightarrow h$ prefactor, a group of m equal orders with r of them having $a_i = 1$ contributes $r + (m - 2r)\nu$; generically, different orders do not cancel systematically, and the ν of the group cancels only when m is even and $r = m/2$. For such M_1 zeros one likewise first moves them away with an analytic regulator in the time power and then runs the recurrences at generic regulator.

In the opposite direction, raising $n_v \rightarrow n_v + 1$, the denominator is

$$\lambda_{0;v,\mathbf{a},\mathbf{r}} = i\kappa_{v,s;\mathbf{a},\mathbf{r}}, \quad \kappa_{v,s;\mathbf{a},\mathbf{r}} = 0. \quad (77)$$

This is the kinematics energy-letter singular surface. Here k_i and q_e are momentum moduli; ordinary vector momentum conservation does not force κ to vanish. When the relevant momenta degenerate to collinear and split into two opposite groups L, R along the axis, momentum conservation becomes $\sum_{\ell \in L} |\mathbf{k}_\ell| - \sum_{\ell \in R} |\mathbf{k}_\ell| = 0$, which is exactly $\kappa = 0$ for the corresponding choice of state signs. In the raising formula, $M_1 = 0$ appears only in the numerator and produces no pole; in the lowering formula, $M_0 = 0$ likewise appears only in the numerator. If both vanish, the row cannot be crossed in either direction and the formulas leave only a lower-sector constraint; one then needs another relation or a different basis, but one still cannot declare that a new master necessarily appears merely from the rank loss. The program should return the direction, shift, state bits, the zero eigenvalue and the original undivided IBP row, without hiding these layers behind a fixed maximum number of steps.

The 0-fold special case is the clearest: $M_1 = \nu_0 + n$ and $M_0 = ip_0$. Lowering fails at $\nu_0 + n = 0$ and raising fails at $p_0 = 0$; these are precisely the two directions to which the two denominators of Eq. (3.59) of [1] belong, not one “unified singular layer.”

For a single massless edge e contracted to $t = s/e$, Eq. (36) directly gives

$$C_{s,s/e}^{(v,e)} = w_e^{(v)} \text{PermuteToGlobalOrder}(|1\rangle_e \otimes I_{\text{spectators}}). \quad (78)$$

If a bundle of full lines $\mathcal{B}$ shares the same theta argument, they are not a product of independent deltas. Sharing means that the two ends of each of the $|\mathcal{B}| \geq 2$ full lines lie in the same two current components, so their theta arguments are all the same $\Delta = \tau_u - \tau_v$; this is exactly the criterion by which the program groups the eligible full lines into bundles. Write

$$\prod_{e \in \mathcal{B}} G_e = \theta(\Delta) \prod_{e \in \mathcal{B}} A_e + \theta(-\Delta) \prod_{e \in \mathcal{B}} B_e, \quad J_e = \frac{A_e + B_e}{2}, \quad D_e = A_e - B_e. \quad (79)$$

Eq. (79) only multiplies out the thetas, and no parity enters at this step: within one bundle $\theta(\Delta)$ is the same function, and since $\theta(\Delta)^2 = \theta(\Delta)$ and $\theta(\Delta)\theta(-\Delta) = 0$, every mixed term vanishes, so the product of $|\mathcal{B}| = n$ lines leaves only two terms instead of 2^n . The 2^n terms arise only for lines with pairwise different theta arguments, and those lines belong to different bundles; the contact events are performed bundle by bundle.

Parity comes from the next step, differentiating the shared theta. The whole bundle has only one θ , so a single differentiation gives only one $\delta(\Delta)$,

$$\partial_\Delta \left[\theta(\Delta) \prod_e A_e + \theta(-\Delta) \prod_e B_e \right] = \delta(\Delta) \left(\prod_e A_e - \prod_e B_e \right), \quad (80)$$

that is, the coefficient of δ is exactly the antisymmetric difference under $A \leftrightarrow B$, in which all the contact information is concentrated; the endpoints are also forced to coincide only once by this single δ , and the components merge only once, independently of how many edges are selected.

The third step telescopes the antisymmetric difference and then changes basis. The telescoping identity is

$$\begin{aligned} \prod_{e=1}^n A_e - \prod_{e=1}^n B_e &= \sum_{k=1}^n (A_1 \cdots A_k B_{k+1} \cdots B_n - A_1 \cdots A_{k-1} B_k \cdots B_n) \\ &= \sum_{k=1}^n D_k \prod_{e < k} A_e \prod_{e > k} B_e, \end{aligned} \quad (81)$$

and each term contains exactly one D_k . Substituting $A_e = J_e + D_e/2$ and $B_e = J_e - D_e/2$ for the spectators and collecting by the number of D factors gives

$$\begin{aligned} \prod_e A_e - \prod_e B_e &= \sum_{S \subseteq \mathcal{B}} \left(1 - (-1)^{|S|} \right) 2^{-|S|} \prod_{e \in S} D_e \prod_{e \notin S} J_e \\ &= \sum_{\substack{\emptyset \neq S \subseteq \mathcal{B} \\ |S| \text{ odd}}} 2^{1-|S|} \prod_{e \in S} D_e \prod_{e \notin S} J_e. \end{aligned} \quad (82)$$

The parity can also be read off from symmetry alone: the left-hand side of Eq. (81) changes sign under $A \leftrightarrow B$, while D_e changes sign and J_e does not, so only terms containing an odd number of D factors can survive; the coefficient $1 - 1$ at $|S| = 0$ removes the empty set automatically, and the same holds for every even $|S|$. In the coefficient $2^{1-|S|}$, the factor $2^{-|S|}$ comes from the $D_e/2$ of each selected edge and the extra factor 2 from $1 - (-1)^{|S|}$ on odd subsets.

The program uses the collected form (82) rather than the term-by-term telescoping (81): the k -th telescoping term corresponds to “contracting only the k -th line,” but after the spectators are expanded the same S receives contributions from several k , so enumerating the telescoping terms would count the same sector transitions repeatedly. After the collection each event appears once and its coefficient is a pure number; in the implementation this means taking the nonempty odd subsets of each bundle, recording the coefficient as $2^{1-|S|}$, and deduplicating by source sector, target sector and the selected line ids.

Hence an odd subset S is a simultaneous contact event: the selected edges are contracted at once, the endpoints merge only once, and the contact selector is the tensor product of the local selectors of the selected edges times $2^{1-|S|}$. If the two ends of an unselected massless full edge become coincident after merging, its odd shared state is zero; an unselected massive full edge uses the equal-time canonical quotient $10 \sim 01$. The program first builds the matrices in the raw binary space and then projects with $P_s M^{\text{raw}} S_s$ onto the true sector masters, so the coincident quotient does not require a general large-matrix inversion.

For an event-reachable component C , the child base power can be written directly from the final partition as

$$A_{C,s} = \sum_{v \in C} A_v + \sum_{e \in E_s(C)} r_e + |C| - 1, \quad (83)$$

where r_e is the raw contact power of the contracted edge. The last term $|C| - 1$ is the number of vertex merges that actually occur; a common-theta event of k edges contributes once, not the erroneous k times. Eq. (83) depends only on the final component and the contracted set, so

different legal contact paths give the same base power. Every event strictly reduces the number of components, so the sector BFS terminates automatically, and a repeated contact of an active self-edge is excluded.

9 The full-graph tensor formula for the dlog DE

Define the diagonal matrix-valued logarithm of each vertex

$$\tilde{\Omega}_{0;v,s} = \text{diag}_{\mathbf{a},\mathbf{r}} [-i \log \kappa_{v,s;\mathbf{a},\mathbf{r}}]. \quad (84)$$

The explicit momentum terms of [1] come only from the massive endpoint slots, which are still two-dimensional h systems:

$$(\Omega_{\text{ex},s})_{\mathbf{ba}} = \delta_{\mathbf{ba}} \left[- \sum_{i \in H(s)} a_i (1 - 2\nu_i) \log k_i \right]. \quad (85)$$

A massless shared-edge slot does not contribute this term, because the default positive prefactor ($s_\nu = +1$, `formulaNu` = $\nu = 1/2$) makes $1 - 2\nu = 0$; under the negative prefactor the $\nu = 1/2$ massless edge is no longer a pure exponential, which was already stated separately after Eq. (11).

If a massless shared edge is instead assigned the next degenerate half-integer order $\nu = 3/2$, the result still lies inside the same family; no new slot, atom, or master is required. From the explicit spherical Hankel functions

$$z^{3/2} H_{3/2}^{(1)}(z) = (1 - iz) z^{1/2} H_{1/2}^{(1)}(z), \quad z^{3/2} H_{3/2}^{(2)}(z) = (1 + iz) z^{1/2} H_{1/2}^{(2)}(z), \quad (86)$$

the $\nu = 3/2$ block exceeds the pure-exponential $\nu = 1/2$ block of Eq. (11) only by a first-degree polynomial in z . After $z = -k\tau$ that polynomial becomes a first-degree polynomial in τ : the constant term reproduces the original $\nu = 1/2$ block; the τ term merely raises the explicit $(-\tau)^A$ power by one, which the package records in the `timeShifts` of the corresponding vertex without introducing a new function type; the accompanying momentum constant ($\pm ik$) is independent of the τ integration and goes directly into the normalization N_s . Therefore a $\nu = 3/2$ massless edge is not a two-dimensional h system either, and contributes no new term to Eq. (85), which reads only the massive endpoint h slots. The general half-integer $\nu = \ell + 1/2$ behaves the same way: $z^{\ell+1/2} H_{\ell+1/2}^{(1,2)}(z)$ is a polynomial of degree ℓ times $e^{\pm iz}$, and term by term reduces to the $\nu = 1/2$ block plus integer powers of τ .

The regular dlog potential of a fixed sector is the full-graph version of Eq. (3.55) of [1]:

$$\Omega_{ss}^{\text{reg}} = \Omega_{\text{ex},s} - i \sum_{v \in V_s} U_s^{-1} \tilde{\Omega}_{0;v,s} U_s M_{1;v,s} (\nu_{0,v} + 1). \quad (87)$$

Its letters are exactly all of Eq. (52); no linear solution of the IBP relations is needed first.

The dlog producer and the iterative producer therefore use the same registry, not a second algorithm: the one-dimensional exponential gives $p_{0,v}$, massive blocks give M_1 , M_0 and Ω_{ex} at the same time, and massless shared blocks give the two endpoints the same state bit, opposite incidence and the contact selector. First construct U_s according to Eq. (49), then write $\log \kappa$ state by state, and finally multiply by $M_1(A + 1)$ to obtain Eq. (87). A general inversion of the dense $2^N \times 2^N$ M_0 is never performed here; the actual inversion is only the letter-by-letter reciprocal of the scalar letters of Eq. (51).

If one changes to user coordinates or Wick coordinates, the DE is not rederived; one only pulls back the differential one-form with Eq. (57). If one changes to the package quotient or the physical-derivative basis, one uses the constant similarity transformation (123) or the gauge transformation (109) containing derivative terms, respectively; the three must not be mixed into a single derivative-free basis change.

The off-diagonal blocks of the DE must use the remaining term of the raising relation, i.e. $\mathcal{R}_{v,s}(\mathbf{e}_v)$ at $\mathbf{n} = \mathbf{e}_v$ of Eq. (64) in this document (denoted $R^{(1)}$ above); one cannot substitute $\mathcal{R}_{v,s}(\mathbf{0})$ from the lowering step (denoted $R^{(0)}$ above) with its lower time powers reduced first, because the two child targets differ by one unit shift and do not commute. The package uses $(-\tau)^A$, whose raising row is

$$M_{0;v,s}\mathbf{F}_s(\mathbf{e}_v) = M_{1;v,s}(\nu_{0,v} + 1)\mathbf{F}_s(0) - \mathcal{R}_{v,s}(\mathbf{e}_v). \quad (88)$$

Hence the contact term in the parameter derivative is $+iM_0^{-1}\mathcal{R}$; when adapting the τ^ν notation of [1] to this form, this overall sign must not be dropped. If

$$\mathcal{R}_{v,s}(\mathbf{e}_v) = \sum_{t \prec s} C_{st}^{(v,1)} \mathbf{F}_t(0), \quad (89)$$

then Eq. (3.68) of [1] generalizes directly to

$$\boxed{\Omega_{st}^{\text{contact}} = +i \sum_{v \in V_s} \chi_v(E_{st}) U_s^{-1} \tilde{\Omega}_{0;v,s} U_s C_{st}^{(v,1)}, \quad t \prec s.} \quad (90)$$

where E_{st} is the set of edges actually selected by that simultaneous event, $N_0(E_{st})$ is the number of masslessFull edges in it, and one defines

$$\chi_v(E_{st}) = \begin{cases} \varsigma_v, & N_0(E_{st}) = 0, \\ (-1)^{N_0(E_{st})}, & N_0(E_{st}) > 0. \end{cases} \quad (91)$$

For pure-massive events, ς_v is the vertex phase sign fixed at initialization; an event containing massless edges also includes the phase of the exponential quotient relative to the h/Wronskian contact atoms. All sign-linear correlations are already contained in $C_{st}^{(v,1)}$: massive endpoints use the complementary-bit selector, and the massless quotient uses the $-2, +2$ of Eq. (119). After differentiating the analytic chamber integral of a single pure-massless full edge with respect to the two vertex energies, Eq. (91) makes the complete DE residual exactly zero component by component; a simultaneous event of three common-theta edges takes $N_0 = 3$. A pure-massive component after contact must have a single ς ; if the input makes the same component mix positive and negative phases, the formula entry fails closed.

9.1 Bit deletion, insertion and the massless quotient in Eq. (3.68) of [1]

First write the pure-massive two-vertex result of Eq. (3.68) of [1], that is, the unreduced form before the extra massless reduction relations are inserted. Let the contracted edge sit at endpoint positions i, j of the left and right vertices, respectively; the parent states are $(\mathbf{a}, \mathbf{b})$ and the child states are $(\hat{\mathbf{c}}, \hat{\mathbf{d}})$. Define

$$K_u^{\text{unreduced}} = T_{n_u}^{-1} \tilde{\Omega}_{0;u} T_{n_u}, \quad K_v^{\text{unreduced}} = T_{n_v}^{-1} \tilde{\Omega}_{0;v} T_{n_v}. \quad (92)$$

Then the two endpoint terms of that equation read component by component

$$\begin{aligned} & \left(\Omega_{s,s/e}^{\text{unreduced}} \right)_{(\mathbf{a}, \mathbf{b}), (\hat{\mathbf{c}}, \hat{\mathbf{d}})} \\ &= -i (K_u^{\text{unreduced}})_{\mathbf{a}, \text{Ins}_i(\hat{\mathbf{c}}, 1-b_j)} (-1)^{b_j} \delta_{\text{Del}_j \mathbf{b}, \hat{\mathbf{d}}} \\ & \quad - i (K_v^{\text{unreduced}})_{\mathbf{b}, \text{Ins}_j(\hat{\mathbf{d}}, 1-a_i)} (-1)^{a_i} \delta_{\text{Del}_i \mathbf{a}, \hat{\mathbf{c}}}. \end{aligned} \quad (93)$$

This is the precise meaning of “deleting the bit of the contracted endpoint and inserting its complementary bit on the other side”; all undeleted bits are preserved by the Kronecker deltas. The $(-\tau)^A$ normalized- J convention of this document converts the overall $-i$ of Eq. (93) into

the $+i\chi_v(E)$ of Eq. (90) according to Eq. (88); the fixed positive pure-massive vertices of [1] take $\chi_v = \varsigma_v = +1$. One must not change only the binary selector and miss this convention transformation.

Now contract a massless edge e . Since the shared slot breaks the vertex-by-vertex factorization, first define the global kernel on the full parent sector

$$\mathcal{K}_{v,s} = U_s^{-1} \tilde{\Omega}_{0;v,s} U_s. \quad (94)$$

In the parent global binary string after the quotient, the edge has a single shared bit, whose position is denoted by p_e ; the child string is denoted by $\hat{c}$. The contact matrix components in the package basis are

$$\left(C_{s,s/e}^{(v,e)} \right)_{\mathbf{d},\hat{c}} = \eta_{s,e} w_e^{(v)} \delta_{d_{p_e},1} \delta_{\text{Del}_{p_e} \mathbf{d},\hat{c}}, \quad (95)$$

where $\eta_{s,e}$ collects the already-absorbed sector-normalization and pinch prefactor, with $w_e^{(u_e)} = -2$ and $w_e^{(v_e)} = +2$. The intermediate-state sum in Eq. (90) then collapses directly:

$$\sum_{\mathbf{d}} (\mathcal{K}_{v,s})_{\mathbf{a},\mathbf{d}} \delta_{d_{p_e},1} \delta_{\text{Del}_{p_e} \mathbf{d},\hat{c}} = (\mathcal{K}_{v,s})_{\mathbf{a},\text{Ins}_{p_e}(\hat{c},1)}. \quad (96)$$

The complete top-to-sub block of a single massless edge is therefore

$$\begin{aligned} \left(\Omega_{s,s/e}^{(e)} \right)_{\mathbf{a},\hat{c}} &= -i\eta_{s,e} \left[w_e^{(u_e)} (\mathcal{K}_{u_e,s})_{\mathbf{a},\text{Ins}_{p_e}(\hat{c},1)} + w_e^{(v_e)} (\mathcal{K}_{v_e,s})_{\mathbf{a},\text{Ins}_{p_e}(\hat{c},1)} \right] \\ &= 2i\eta_{s,e} \left[(\mathcal{K}_{u_e,s})_{\mathbf{a},\text{Ins}_{p_e}(\hat{c},1)} - (\mathcal{K}_{v_e,s})_{\mathbf{a},\text{Ins}_{p_e}(\hat{c},1)} \right]. \end{aligned} \quad (97)$$

Thus one does not keep two independent 2×2 endpoint indices and then sum: the $4 \rightarrow 2$ quotient has already turned them into one shared bit; the so-called “sum over the two endpoints” is only the addition of the two component log kernels in Eq. (97).

If another massless edge $f \neq e$ is not contracted, its shared bit is a spectator and automatically gives

$$\delta_{r_f^{\text{parent}}, r_f^{\text{child}}}. \quad (98)$$

in Eq. (95). It does not produce an extra sum over two endpoint bits; it only continues to enter the energy letters and $\mathcal{K}_{v,s}$ of the two endpoints with opposite incidence signs. In the mixed massive/massless case, one only needs to assemble the massive endpoint bits and the other shared bits into the same global string according to Eq. (60); after deleting p_e , all remaining bits are preserved by the second delta of Eq. (95).

If a generalized or multi-edge contact makes $R^{(1)}$ land on a nonzero shift $\mathbf{q}$ of the child, the program now calls the same recurrence formula child state by child state and constructs

$$\mathbf{F}_t(\mathbf{q}) = \mathcal{Q}_t(\mathbf{q} \rightarrow 0) \mathbf{F}_{\text{all}}(0), \quad \Omega_{s \rightarrow *}^{(v)} = -\mathcal{K}_{v,s} C_{st}^{(v,1)} \mathcal{Q}_t(\mathbf{q}_v \rightarrow 0). \quad (99)$$

Here $\mathbf{F}_{\text{all}}(0)$ is ordered according to the global master-integral order of the context, so $\mathcal{Q}_t$ is allowed to continue down to the lower contact sectors of t , rather than keeping only the direct child. The program stores, column by column, the residual, the unresolved shifted integrals and the direction-sensitive singular layers. Only when all columns close is Eq. (99) written into the connection; otherwise it returns

`contactShiftReductionFailed.`

An ordinary one-edge contact still gives $\mathbf{q} = 0$ directly by the base-power choice.

After ordering the sectors by the number of remaining full edges, the complete connection is block triangular:

$$\text{d} \begin{pmatrix} \mathbf{F}_{s_0} \\ \mathbf{F}_{s_1} \\ \vdots \end{pmatrix} = \text{d} \begin{pmatrix} \Omega_{s_0 s_0} & \Omega_{s_0 s_1} & \cdots \\ 0 & \Omega_{s_1 s_1} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} \begin{pmatrix} \mathbf{F}_{s_0} \\ \mathbf{F}_{s_1} \\ \vdots \end{pmatrix}. \quad (100)$$

Whether the matrix is written as upper or lower triangular depends only on the sector ordering; the physical content is that contact only points from a parent to a strictly lower sector.

10 The dimensionless h equation, physical quadratic-momentum terms and realification

Eq. (12) is the equation to be used inside the building block:

$$\partial_z^2 h_0 = -h_0. \quad (101)$$

Only restoring the physical time derivative with Eq. (10) gives

$$\boxed{\partial_\tau^2 h_0 = k^2 \partial_z^2 h_0 = -k^2 h_0.} \quad (102)$$

So $-k^2$ is not a second set of EOMs; it is the physical coefficient of the dimensionless constant -1 after two chain-rule steps. Continuing to use the two states h_0, h_1 internally allows the 2×2 formulas of [1] to be reused unchanged.

If the backend realification uses

$$k = -i\kappa, \quad (103)$$

then

$$-k^2 = +\kappa^2. \quad (104)$$

If the program token for the real variable happens to be named $i\kappa$, the $i\kappa^2$ in the output means the square of this real variable, not the imaginary unit i times k^2 .

Now distinguish the h-state convention of [1], called the reference h-state below, from the physical endpoint derivatives. Let

$$\mathcal{F}_{ab} = \partial_{\tau_u}^a \partial_{\tau_v}^b G_e \quad (a, b = 0, 1) \quad (105)$$

denote the physical states from which $-k$ has not been divided out. The regular top-sector part satisfies

$$\mathcal{F}_{01} = -\mathcal{F}_{10}, \quad \mathcal{F}_{11} = k^2 \mathcal{F}_{00}, \quad (106)$$

while the second relation of the complete full propagator must additionally receive the contact distribution from the theta derivative, which is exactly the lower sector to which Eq. (78) belongs.

If one chooses the physical two-state $\mathbf{g}_{\text{phys}} = (\mathcal{F}_{00}, \mathcal{F}_{10})^T$, its relation to the reference h two-state is

$$\mathbf{g}_{\text{phys}} = N(k) \mathbf{g}_h, \quad N(k) = \begin{pmatrix} 1 & 0 \\ 0 & -k \end{pmatrix}. \quad (107)$$

The regular embedding of the physical four states is therefore

$$\begin{pmatrix} \mathcal{F}_{00} \\ \mathcal{F}_{01} \\ \mathcal{F}_{10} \\ \mathcal{F}_{11} \end{pmatrix}_{\text{reg}} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ k^2 & 0 \end{pmatrix} \mathbf{g}_{\text{phys}}. \quad (108)$$

Hence if the final DE uses the physical two-state rather than the reference h two-state, one must perform

$$\boxed{\mathcal{A}_{\text{phys}} = dN N^{-1} + N \mathcal{A}_h N^{-1}, \quad dN N^{-1} = \begin{pmatrix} 0 & 0 \\ 0 & d \log k \end{pmatrix}.} \quad (109)$$

It is recommended that the formula producer always keep the dimensionless h-state basis internally, so that S_e, P_e, T, U_s are all kinematics-independent constants; only when outputting the physical-derivative states should $N(k)$ and Eq. (109) be applied.

11 Interface with the package conventions

11.1 The massless quotient basis of the package

The F_{ab} above are the reference h-states. In order to separate the momentum factor of each endpoint derivative from the discrete states, the package uses another algebraic quotient. For an ordered edge $e = (u_e, v_e)$, let $\Delta = \tau_{u_e} - \tau_{v_e}$, and use $\sigma_e = +1, -1$ for G^{++}, G^{--} respectively; define

$$\mathbf{m}_{e,0} = \theta(\Delta)e^{-i\sigma_e q_e \Delta} + \theta(-\Delta)e^{+i\sigma_e q_e \Delta}, \quad (110)$$

$$\mathbf{m}_{e,1} = -\theta(\Delta)e^{-i\sigma_e q_e \Delta} + \theta(-\Delta)e^{+i\sigma_e q_e \Delta}. \quad (111)$$

The shared two-state and the four-state embedding of the package are

$$\mathbf{g}_e^J = (\mathbf{m}_{e,0}, \mathbf{m}_{e,1})^T, \quad \begin{pmatrix} F_{00}^J \\ F_{01}^J \\ F_{10}^J \\ F_{11}^J \end{pmatrix} = S_e^J \mathbf{g}_e^J, \quad S_e^J = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 0 & 1 \\ -1 & 0 \end{pmatrix}. \quad (112)$$

So in the package one indeed has

$$F_{01}^J = -F_{10}^J, \quad F_{11}^J = -F_{00}^J. \quad (113)$$

This minus sign does not conflict with Eq. (28): F_{ab}^J has already factored out the $\pm i\sigma_e q_e$ of the endpoint derivatives, so it is not $h_a h_b$. Under a common kinematics-independent normalization, the two shared two-states satisfy

$$\mathbf{g}_e^J = D_{\sigma_e} \mathbf{g}_e^h, \quad D_{\sigma_e} = \text{diag}(1, i\sigma_e). \quad (114)$$

Therefore the $\mp i q_e \sigma_2$ of the two endpoints in the reference h-state become, in the package quotient, exactly

$$Q_{u_e,e}^J = D_{\sigma_e}(-i q_e \sigma_2) D_{\sigma_e}^{-1} = +i \sigma_e q_e \sigma_1, \quad (115)$$

$$Q_{v_e,e}^J = D_{\sigma_e}(+i q_e \sigma_2) D_{\sigma_e}^{-1} = -i \sigma_e q_e \sigma_1, \quad \sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}. \quad (116)$$

The equivalent complete endpoint rules are

$$\partial_{\tau_{u_e}} \mathbf{m}_{e,n} = +i \sigma_e q_e \mathbf{m}_{e,1-n} - 2n \delta(\Delta), \quad (117)$$

$$\partial_{\tau_{v_e}} \mathbf{m}_{e,n} = -i \sigma_e q_e \mathbf{m}_{e,1-n} + 2n \delta(\Delta). \quad (118)$$

This simultaneously gives the contact weights of the ordered first and second endpoints in the package quotient

$$w_{e,J}^{(u_e)} = -2, \quad w_{e,J}^{(v_e)} = +2. \quad (119)$$

They agree exactly with Eq. (37), because

$$D_{\sigma_e} |1\rangle_e (+2i\sigma_e) = -2|1\rangle_e, \quad D_{\sigma_e} |1\rangle_e (-2i\sigma_e) = +2|1\rangle_e. \quad (120)$$

A cycle line stores each factor of q_e as $b_e \rightarrow b_e - 1$; a fixed/non-loop line has no b_e slot and must keep the structured modulus q_e explicitly. Two regular derivatives at the same endpoint therefore give $-q_e^2$, while one at each endpoint gives $+q_e^2$.

The diagonalizing matrix of the package quotient can be obtained directly from the matrix of [1]:

$$T_e^J = T D_{\sigma_e}^{-1}, \quad T_e^J Q_{u_e,e}^J (T_e^J)^{-1} = T(-i q_e \sigma_2) T^{-1}. \quad (121)$$

So the two bases give the same set of energy letters; σ_e only swaps the label of the shared bit. If one instead uses the fixed real Hadamard matrix, this swap appears explicitly as $\sigma_e(2r_e - 1)q_e$ in the letter.

For the whole sector, embed Eq. (114) into every uncontracted massless full-edge slot and denote the resulting constant matrix by

$$B_s = \bigotimes_{e \in E_m(s)} D_{\sigma_e}, \quad (122)$$

with identity matrices on the remaining slots. All regular and contact blocks can then be transformed directly as

$$M_{0;v,s}^J = B_s M_{0;v,s}^h B_s^{-1}, \quad \Omega_{ss}^J = B_s \Omega_{ss}^h B_s^{-1}, \quad C_{st}^J = B_s C_{st}^h B_t^{-1}. \quad (123)$$

The last equation is a basis change between spaces of different dimensions, not a similarity transformation; when $s \rightarrow t$ deletes the edge e , the left multiplication by D_{σ_e} exactly accomplishes Eq. (120).

11.2 The unique quotient and its four-state derivation space

MadStree uniquely uses one shared two-state `masslessShared` slot for every `masslessFull`. The public input has no representation-selection option; recurrence, contact, dlog and numerical layers consume only the quotient. The ordered-endpoint four-state space (00, 01, 10, 11) below is used only to derive that quotient and is not a second program context, slot registry or formula route.

For the derivation, work first in a four-state space with the constant normalization compatible with the quotient, without introducing the $2/\pi$ of the bare Hankel product. Using S_e, P_e of Eq. (30), define

$$S_{h \leftarrow J} = S_e D_\sigma^{-1}, \quad P_{J \leftarrow h} = D_\sigma P_e, \quad P_{J \leftarrow h} S_{h \leftarrow J} = I_2. \quad (124)$$

For G^{++} this gives

$$S_{h \leftarrow J} = \begin{pmatrix} 1 & 0 \\ 0 & i \\ 0 & -i \\ 1 & 0 \end{pmatrix}, \quad P_{J \leftarrow h} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & i & 0 \end{pmatrix}. \quad (125)$$

The contact column of the first endpoint in the four-state derivation space is $2i\sigma(0, -1, 1, 0)^T$, and the second endpoint takes the opposite sign; after projecting with $P_{J \leftarrow h}$, one obtains exactly the $-2, +2$ of Eq. (119). For a sector with spectator slots, Eq. (124) shows how the four-state space is projected once into the shared production two-state space. The program then generates M_1, M_0 , the iterative reduction and the complete block-triangular dlog only in the quotient space and retains no four-state production branch.

11.3 $(-\tau)^A$ and normalized- J

The previous text used the τ^{ν_0} convention of [1]. The integral object used by recurrence, DEs and numerical transport is the normalized object

$$J_s = \mathcal{N}_s I_s, \quad (126)$$

where the bare integral with the same existing indices is defined compactly by

$$I_s(\mathbf{n}; \mathbf{a}) = \int_{(-\infty, 0)^{c_s}} \prod_{C=1}^{c_s} d\tau_C (-\tau_C)^{A_{C,s} + n_C} e^{ip_{C,s} \tau_C} \mathcal{H}_{s,\mathbf{a}}(\boldsymbol{\tau}). \quad (127)$$

Here $s = \text{sectorKey}$, $\mathbf{n} = \text{aShifts}$ and $\mathbf{a} = \text{stateBits}$; $\mathcal{H}_{s,\mathbf{a}}$ is the ordered product of the active-line two-state building blocks defined above. Massive endpoints read their own state bits, whereas a massless shared quotient reads one bit shared by the whole edge. The $++/--$ time ordering of a Full line stays inside its building block, and contracted lines are absent. The program does not expand this product separately for every master.

The single-integral query is

`MSIntegralDefinition[MSIntegral[s,n,a], context]`

It returns the same-index `MSBareIntegral[s,n,a]`, the exact $\mathcal{N}_s$, and an inert equality; each ordered record returned by `MSMasterIntegrals[context]` contains the same fields. For $\mathcal{N}_s = 1$ the displayed relation is $J_s(\mathbf{n}; \mathbf{a}) = I_s(\mathbf{n}; \mathbf{a})$. The time power is written as $(-\tau_v)^{A_{v,s}}$, so

$$\partial_{\tau_v}(-\tau_v)^{A_{v,s}} = -A_{v,s}(-\tau_v)^{A_{v,s}-1}. \quad (128)$$

In the same h-state basis, the package adapter therefore writes Eq. (66) of this document exactly as

$$-M_{1;v,s}(A_v)\mathbf{J}_s(\mathbf{a} - \mathbf{e}_v) + M_{0;v,s}\mathbf{J}_s(\mathbf{a}) + \mathcal{R}_{v,s}^J(\mathbf{a}) = 0, \quad (129)$$

where the minus sign comes entirely from Eq. (128). If the package quotient basis is used, one only needs to apply the constant similarity transformation of Eq. (114) to M_0 on each massless shared slot; M_1 does not contain that slot. This adaptation does not change the shared-edge slot registry.

Under the package conventions the two one-step formulas are therefore explicitly

$$\mathbf{J}_s(\mathbf{n} - \mathbf{e}_v) = M_{1;v,s}(A_v + n_v)^{-1} [M_{0;v,s}\mathbf{J}_s(\mathbf{n}) + \mathcal{R}_{v,s}^J(\mathbf{n})], \quad (130)$$

$$\mathbf{J}_s(\mathbf{n} + \mathbf{e}_v) = M_{0;v,s}^{-1} [M_{1;v,s}(A_v + n_v + 1)\mathbf{J}_s(\mathbf{n}) - \mathcal{R}_{v,s}^J(\mathbf{n} + \mathbf{e}_v)]. \quad (131)$$

This preserves the directionality of Eqs. (76)–(77); the overall sign change does not alter which matrix is inverted in that direction.

In the normalized basis one must also use

$$\partial_x J_s = (\partial_x \log \mathcal{N}_s) J_s + \mathcal{N}_s \partial_x I_s, \quad C_{st}^J = \frac{\mathcal{N}_s}{\mathcal{N}_t} C_{st}^I. \quad (132)$$

So the sector diagonal blocks receive an additional $d \log \mathcal{N}_s I_s$, and the off-diagonal contact blocks are multiplied by $\mathcal{N}_s/\mathcal{N}_t$. Only when this ratio has been turned into a kinematics-independent constant by the basis normalization is Eq. (90) directly a constant-residue dlog; otherwise an additional basis/gauge transformation is needed, and one cannot claim completion merely from the literal $\log \kappa$.

11.4 The forward and inverse transformations of the H/h state vectors

Let $p = s_\nu \nu$. The local relation between the same function h and the bare Hankel states is

$$\begin{pmatrix} h(\nu, 0; z) \\ h(\nu, 1; z) \end{pmatrix} = T_{H \rightarrow h}(p, z) \begin{pmatrix} H_\nu(z) \\ \partial_z H_\nu(z) \end{pmatrix}, \quad T_{H \rightarrow h}(p, z) = \begin{pmatrix} z^p & 0 \\ pz^{p-1} & z^p \end{pmatrix}. \quad (133)$$

Its inverse matrix is

$$T_{h \rightarrow H}(p, z) = \begin{pmatrix} z^{-p} & 0 \\ -pz^{-p-1} & z^{-p} \end{pmatrix}, \quad T_{h \rightarrow H} T_{H \rightarrow h} = I_2. \quad (134)$$

MadStree's `MSHTohMatrix[nu,z,context]` and `MShToHMatrix[nu,z,context]` implement these two matrices directly. `MSConvertBasis` can also take their Kronecker product according to the slot registry of a fixed sector: each massive endpoint uses its own $z = -k\tau_{\text{component}}$, while the massless shared quotient is not a bare Hankel endpoint pair and therefore multiplies by identity

in this basis change. `NuConvention` is chosen only at `MSInitTree`; after the context is built, both the local matrices and the full-sector basis change read that value fixedly and do not accept per-call overrides. The forward and inverse transformations have been checked separately for both positive/negative prefactor contexts to give the identity.

What is transformed here is the same-ordered state vector of the integrand, not the already-integrated `MSIntegral`. The reason is that the z^p and z^{p-1} of Eq. (133) also change the component base time power at the same time; the first version has not built an independent H-family sector metadata, so overloading the integral object explicitly fails closed, and this step cannot be disguised as merely flipping a state bit.

The public time-only representation of dSIBP is `J[sectorKey,timeShifts,stateBits]`, which corresponds slot by slot to MadStree's `MSIntegral[sectorKey,timeShifts,stateBits]`; `stateBits` are discrete states of the n_i type in the full-sector registry and are not required to be grouped by vertex. This correspondence only converts the integral identity and does not replace the normalization: both contexts must use the same $\mathcal{N}_s$ for every sector, and differentiation must include the $\partial \log \mathcal{N}_s$ of Eq. (132). The default massless input of MadStree is $\nu = 1/2$ with positive prefactor; in this case `formulaNu` = $s_\nu \nu = +1/2$, and the code and the formulas of this document agree slot by slot on the same $+\nu$ base formula. With `NuConvention -> "Negative"` one has `formulaNu` = $-1/2 = \nu_{\text{ref}}$, used only to compare verbatim with the printed formulas of that paper. This convention does not replace line metadata, theta, or contact normalization.

11.5 Single-vertex families, local inverses and the complete reduction interface

A single-vertex family does not need to construct a fake graph. The compact input is

```
ctx = MSInitVertexFamily[<|
  "ki" -> {k0,k1,...},
  "nui" -> {A0,nui,...},
  "hankelBranches" -> {1,2,...}
|>, NuConvention -> "Positive"];
```

Here `First[ki]` and `First[nui]` are respectively the parameter of the theta-free exponential attached to the vertex and the base power of $(-\tau)$; the remaining entries give the momentum, the ν used as-is as the Hankel order, and the Hankel branch of each h block. The explicit data model uses `externalLegEnergy/timePower/hBlocks/exponentialBlocks`. `exponentialBlocks` is only a convenience of the input layer: at initialization its entries are folded into the effective external-leg energy with their literal `exponentType` signs, after which only the single parameter of Eq. (5) remains and no independent slot is created, while each h block is a two-dimensional massive endpoint slot. The context can directly call `MSFormulaMatrices`, `MSMasterIntegrals`, `MSDLogDE`, and `MSBoundaryData`, then use `MSEvaluatePath` for the one-stage numerical workflow. `NuConvention` is chosen only at initialization and all later interfaces read it from the context.

The complete explicit model has the shape

```
ctx = MSInitVertexFamily[<|
  "externalLegEnergy" -> k0,
  "timePower" -> A0,
  "hBlocks" -> {
    <|"id"->h1,"momentum"->k1,"nu"->nui,"hankelBranch"->1|>
  },
  "exponentialBlocks" -> {
    <|"id"->e1,"momentum"->p1,"exponentType"->"-"|>
  },
  "vertexType" -> "+",
  "normalization" -> 1
|>;
```

Here `vertexType` fixes the SK contour branch of the whole vertex, while `exponentType` is only the literal sign of that extra exponential block in the effective external-leg energy. A single-vertex family has no propagator incidence, so its initialization schema intentionally differs from tree `vertices/lines`. Once the context exists, both forms use the same masters, recurrence, DE, boundary, numerical options, and return structure.

The implementation contains no general $2^p \times 2^p$ symbolic inverse. For a massive endpoint, the local non-trivial direction of M_0 is σ_2 , and one directly uses the constant matrix T of [1] with its explicit inverse; for a massless shared edge, the local direction is σ_1 , and one uses the self-inverse Hadamard matrix

$$H_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H_2^{-1} = H_2. \quad (135)$$

Pure exponential factors are one-dimensional identity matrices. The full-sector U_s and U_s^{-1} are obtained only as Kronecker products of these local matrices. Since M_1 is already diagonal in the state-bit basis, `msM1Inverse` only computes

$$\text{diag}(M_1^{-1})_r = \lambda_{1,r}^{-1}; \quad (136)$$

while M_0 is computed only in the common diagonal basis as

$$M_0^{-1} = U_s^{-1} \text{diag}((i\kappa_r)^{-1}) U_s. \quad (137)$$

Thus after the massless $4 \rightarrow 2$ quotient the local-diagonalization inversion still applies unchanged. The necessary condition is that all local operators on the same slot contain only the same Pauli direction; if non-collinear directions appear in the future, the program must reject Eq. (137) instead of pretending they can be simultaneously diagonalized.

The complete reduction entry point is

```
MSReduce[linearCombination, context, MasterBasis -> Automatic]
```

It recursively moves each component shift to zero for every `MSIntegral` according to Eqs. (130)–(131), and continues to reduce the lower sectors along the contact DAG; repeated subproblems are reused through a memo table. The key returned fields are

```
result, masterBasis, masterRules, coefficientVector,
nonMasterResidual, remainingShiftedIntegrals,
memoizedIntegralCount, singularLayers
```

where the master basis, the coefficient vector and the rules are strictly in the same order, and `result` is the explicit linear combination of master integrals. The status is `reduced` only when the residual is zero and the shifted-integral list is empty. `MasterBasis` may be any permutation of all context masters, but it cannot miss or repeat entries. The singular layers record, for each step, the direction, the component and the denominator that was actually inverted: toward zero with a positive shift only the energy letters of M_0 are reported; with a negative shift only the M_1 eigenvalues are reported. This is exactly the program contract of Eqs. (76)–(77).

The same entry directly accepts `List`, regular `Table`, empty lists and ragged nested lists. The returned `result`, per-element `coefficientVector/masterRules`, residuals and remaining shifts preserve the input shape; all elements share one recursion memo. A failed element reports its exact list position and does not mark the partial results of other elements as a complete reduction.

11.6 Small-time endpoints and the safe time-IBP shift

Integer time-IBP shifts may not cross a divergent endpoint. `MadStree` does not solve a new local series here. It substitutes only the known two leading small-time powers of h into every reachable component and enumerates all binary branch combinations of that component. For a zero-shift

Gamma argument α , if $\alpha \in \{0, -1, -2, \dots\}$, the original master diverges and initialization is rejected immediately. If $\alpha = m \in \mathbb{Z}_{>0}$, initialization is allowed, but that branch is restricted to

$$n \geq 1 - m, \quad (138)$$

because the next shift reaches a non-positive integer. `MSRecurrenceStep`, `MSReduce`, and shifted-child reduction in the dlog reject only an actual request that crosses this bound, returning the original vertices, sector, local branch, α , and the requested shift. An α containing a symbolic analytic regulator remains symbolic and is not misclassified as an integer.

12 Large pure-exponential-energy boundary and blow-up for arbitrary time-only graphs

12.1 Damping variables and auxiliary one-dimensional exponentials

For each root vertex v , choose the pure vertex exponential of Eq. (53) and use the internal positive variable K_v of Eq. (55). Since $\tau_v = -t_v < 0$, both kinds of vertices satisfy

$$\exp(i\varsigma_v k_{0,v} \tau_v) = \exp(-K_v t_v), \quad K_v > 0. \quad (139)$$

So $K_v \rightarrow \infty$ localizes the integral to $\tau_v \rightarrow 0^-$. This is the same as the single-vertex family boundary of Sec. 3 of [2]; that reference uses the chart with $\varsigma_v = +1$ and $k_{0,v} \rightarrow -i\infty$.

If the original integrand has no pure vertex exponential at some vertex, one may add an auxiliary energy $\exp(i\varsigma_v k_{0,v}^{\text{aux}} \tau_v)$. It only maps

$$p_{0,v} \mapsto p_{0,v} + \varsigma_v k_{0,v}^{\text{aux}}, \quad M_{0;v,s} \mapsto M_{0;v,s} + i\varsigma_v k_{0,v}^{\text{aux}} I_s. \quad (140)$$

Hence under generic kinematics it does not increase the number of slots or masters and does not change M_1 . However, $K_v^{\text{aux}} = 0$ is a specialization of this augmented family; whether the point is still ordinary must be decided by the full-sector letters and cannot be inferred from “a one-dimensional exponential does not increase the masters.”

12.2 Strict total time order, nested blow-up and theta fixing

Directly choose a strict total order for all root times, and take the opposite strict order of damping energies

$$K_{\sigma(1)} \gg K_{\sigma(2)} \gg \dots \gg K_{\sigma(V)} \gg 1. \quad (141)$$

This step does not require the propagator incidence graph to be a tree. What this document calls a time-only loop graph is a graph that is allowed to contain loops, but where all line momenta are still treated as fixed parameters and only the time integrals of the vertices are performed; it does not include loop-momentum integration. A strict total order orders every pair of times, and for non-adjacent vertices the order merely selects one blow-up chart.

Define the nested coordinates

$$x_j = \frac{K_{\sigma(j+1)}}{K_{\sigma(j)}} \quad (1 \leq j < V), \quad x_V = \frac{1}{K_{\sigma(V)}}. \quad (142)$$

Then

$$\frac{1}{K_{\sigma(j)}} = \prod_{r=j}^V x_r, \quad x_1, \dots, x_V \longrightarrow 0. \quad (143)$$

For two vertices with $K_1 \gg K_2$, this is the $x = K_2/K_1$, $y = 1/K_2$ of Sec. 4.1 of the reference. In the original inverse variables, the three divisors $t_1, t_2, t_1 + t_2$ pass through the two-dimensional origin simultaneously, a degenerate intersection; Eq. (142) resolves it analytically into a normal-crossing chart.

The exponential localization scales satisfy

$$|\tau_{\sigma(1)}| \ll |\tau_{\sigma(2)}| \ll \cdots \ll |\tau_{\sigma(V)}| \ll 1. \quad (144)$$

Since all τ approach zero from the negative side, large K corresponds to small $|\tau|$; hence $K_u \gg K_v$ implies $\tau_u > \tau_v$, and therefore

$$\theta(\tau_u - \tau_v) \longrightarrow 1, \quad \theta(\tau_v - \tau_u) \longrightarrow 0. \quad (145)$$

For G^{++} or G^{--} , Eq. (145) must be applied to every explicit theta in the propagator expansion according to the order of its argument; one cannot guess a single value from the SK label alone. All thetas in an arbitrary time graph therefore become 0 or 1. If some term requires a cyclic inequality, for example

$$\theta(\tau_1 - \tau_2)\theta(\tau_2 - \tau_3)\theta(\tau_3 - \tau_1), \quad (146)$$

then it is automatically zero in every strict-total-order chart, instead of producing a new boundary region.

12.3 Choosing the preferred rank from a user-preferred ordinary point

The user may additionally provide a preferred starting point P_* . The point is still submitted in the user variables of initialization, including all $k_{0,v}$ and all propagator moduli q_e ; the package first computes with the coordinate map of Eq. (57)

$$K_v(P_*) = i\varsigma_v k_{0,v}(P_*). \quad (147)$$

If $\text{Re } K_v(P_*) > 0$, the preferred strict chart takes

$$\text{Re } K_{\sigma(1)}(P_*) > \cdots > \text{Re } K_{\sigma(V)}(P_*), \quad (148)$$

with ties broken deterministically first by $|K_v(P_*)|$ and then by the root vertex order. Note that the order must not be reversed: a large K_v supports only a smaller typical $t_v = |\tau_v|$ to avoid the exponential suppression of $e^{-K_v t_v}$, so Eq. (148) corresponds to

$$|\tau_{\sigma(1)}| \ll \cdots \ll |\tau_{\sigma(V)}|, \quad \tau_{\sigma(1)} > \cdots > \tau_{\sigma(V)}. \quad (149)$$

This choice makes the scale hierarchy of the boundary chart closest to P_* , usually shortening the first continuation segment and possibly reducing the letter hypersurfaces that must be bypassed; it is not a theorem of “globally fewest singularities.” The program must still issue a normal-crossing certificate for the complete top/subsector connection of the candidate chart, and verify that the path from an ordinary anchor inside the chart to P_* does not cross any letter. If some $\text{Re } K_v(P_*) \leq 0$, that point cannot directly define a damping rank; it may still be used as a later ordinary point, but the preferred chart is then given explicitly by the user or determined by another positive-damping anchor.

12.4 Contact subsectors inherit the same root rank

In a sector s , the delta contact merges a set of root vertices into a component C . Its damping energy and rank are defined as

$$K_C = \sum_{v \in C} K_v, \quad r_C = \max_{v \in C} r_v. \quad (150)$$

The strict total order guarantees that K_C is dominated by the unique largest-rank root in C . The delta merge changes the component base time power and the sector normalization, but does not reselect a subsector rank; when a remaining theta connects two components, only their dominant roots are compared.

Let v_C be the dominant root in C . The boundary form of any letter of Eq. (52) is

$$\begin{aligned}\kappa_{C,s;\mathbf{a},\mathbf{r}} &= -iK_C + E_{C,s;\mathbf{a},\mathbf{r}} \\ &= -iK_{v_C} \left[1 + \text{higher monomials in } x + \frac{iE_{C,s;\mathbf{a},\mathbf{r}}}{K_{v_C}} \right],\end{aligned}\tag{151}$$

where the bracket equals 1 at $x = 0$. All energy letters of every contact-reachable sector are therefore a coordinate monomial times a boundary-nonzero unit; this is the direct resolution of the linear letters of arbitrary time graphs by the strict-rank blow-up.

This conclusion is in fact more general than the component letters. The diagonal blocks of a fixed sector contain only the $\log \kappa$ of Eq. (84). The top-to-sub formula (90) merely multiplies the same $\tilde{\Omega}_{0;v,s}$ of the parent by the kinematics-independent contact map $C_{st}^{(v,1)}$, so no new $\log$ argument is produced. If $R^{(1)}$ does not land directly on the child master and further reduction is needed, the new denominators come only from the M_0^{-1} of the child and are still energy letters of the same type of the child component. Inducting along the strictly descending contact DAG, every denominator containing K in the complete block-triangular DE has the form

$$D_{\alpha,s}(K,q) = Q_{\alpha,s}(q) + \sum_{v=1}^V c_{\alpha,s;v} K_v, \quad \text{supp } c_{\alpha,s} \neq \emptyset, \tag{152}$$

where $c_{\alpha,s;v}$ is kinematics-independent. For an ordinary component letter, c_v is exactly the indicator of the component; more general constant contact bases or SK phases only change the nonzero constant coefficients.

Let

$$j_\alpha = \min\{j \mid c_{\alpha,s;\sigma(j)} \neq 0\} \tag{153}$$

be the first position of the support of this denominator in the strict energy total order. Substituting Eq. (142), it factorizes strictly as

$$\begin{aligned}D_{\alpha,s} &= K_{\sigma(j_\alpha)} \left[c_{\alpha,s;\sigma(j_\alpha)} + \sum_{j>j_\alpha} c_{\alpha,s;\sigma(j)} \prod_{r=j_\alpha}^{j-1} x_r \right. \\ &\quad \left. + Q_{\alpha,s} \prod_{r=j_\alpha}^V x_r \right].\end{aligned}\tag{154}$$

The bracket equals the nonzero constant $c_{\alpha,s;\sigma(j_\alpha)}$ at $x = 0$. All strict transforms of such denominators are therefore units; their pullback divisors consist only of the coordinate hyperplanes of $x_{j_\alpha}, \dots, x_V$. A hyperplane that recurs is the same irreducible divisor and does not increase the number of normals, so near the boundary there are at most V mutually independent coordinate normals. This proves: as long as the denominators of the dlog and top-to-sub formulas keep the affine-linear structure of Eq. (152), the blow-up corresponding to a strict ordering of all τ_i eliminates the degenerate singularities caused by all $K_i \rightarrow \infty$ occurring simultaneously. The proof is independent of whether the original propagator graph is a tree or a time-only loop graph.

The program must still provide a certificate and cannot rely only on the above formal check of the top sector. A simple and sufficient algorithm is: collect all irreducible divisors of the complete block-triangular connection, the normalization gauge and the coordinate Jacobian; substitute Eq. (142); extract all monomial powers of the x_j ; and require the remaining strict transforms to be nonzero at $x = 0$. If some strict transform still vanishes, check, for the deduplicated local defining functions $f_1, \dots, f_m$,

$$\nabla f_i \neq 0, \quad \text{rank} \left(\frac{\partial(f_1, \dots, f_m)}{\partial(x_1, \dots, x_V)} \right) = m \leq V. \tag{155}$$

If Eq. (155) is not satisfied, the singularity is still a degenerate multi-variable one. For the letters of Eq. (152), Eq. (154) already proves that all strict transforms become units; the check in the

program is a certificate for the formula implementation and for the extra gauge/normalization, not a second guess of this conclusion.

The public interface `MSBoundaryChartCertificate` implements this certificate. It collects all sector letters of `MSDLogDE`, the normalization divisors and the shifted-contact recurrence singular layers, and returns for each divisor the dominant rank, the coordinate monomial, the strict transform and its value at $x = 0$; it also returns the complete Jacobian from the user energies to K_v and then to x_j , the per-sector theta-fixing table, and the $P_s S_s = I$ quotient check. `RankOrder->All` enumerates all root strict charts. `MSBoundaryData` forces consumption of the certificate of the chosen chart before constructing the infinity Frobenius data and the finite matching points; any non-affine new denominator, vanishing strict transform, zero Jacobian or uncertified quotient returns `BoundaryChartNotCertified` and does not continue the numerical transport.

The theorem above also fixes the scope of automation. Singularities that contain no K_v at all are already listed explicitly in the DE letters; they are not obstacles for this boundary producer. An ordinary polyline can use "Avoid" to refuse a crossing. For an explicitly supplied finite singular user point, "Automatic" uses the backend power-log basis to classify a finite limit, a pole, or a logarithmic divergence and uses the same local basis for user points on both sides inside its convergence disk. More general blow-ups, non-collinear turning at a singularity, and new denominators introduced by an extra gauge transformation do not automatically inherit the proof above.

The boundary condition of each sector must treat that sector itself as the top sector: use the post-contact component base power, the normalization and Eq. (150). Compute the bottom sectors first, then move up along the sector DAG. After theta fixing, the homogeneous leading term factorizes into a product of component vertex-family boundary coefficients; but the parent may also receive a leading contribution from the lower-sector solution. In the five-dimensional example of Sec. 4.2 of the reference, the remaining integral already has nonzero leading coefficients on the second and third components of the original sector, which is exactly why writing only the homogeneous product misses terms. A general producer should solve the block-triangular Frobenius leading system, or equivalently keep the endpoint expansions of the theta/delta, to obtain the complete same-ordered boundary vector recursively.

12.5 Controlled transport of an auxiliary external-leg energy to zero

From Eq. (51),

$$\det M_{0;C,s} \propto \prod_{\mathbf{a},\mathbf{r}} \kappa_{C,s;\mathbf{a},\mathbf{r}}. \quad (156)$$

So the raising recurrence (71) fails on any $\kappa = 0$. The smallest counterexample is a single full edge $e = (u, v)$: the top-sector letters are

$$p_{0,u} \pm q_e, \quad p_{0,v} \pm q_e, \quad (157)$$

which, for generic $q_e \neq 0$, do not vanish even when the two auxiliary energies are set to zero; but the edge-contracted sector has no such two-dimensional slot and necessarily contains

$$\kappa_{\{u,v\}} = p_{0,u} + p_{0,v}. \quad (158)$$

Therefore, when these two p_0 consist only of the auxiliary energies to be removed, the complete top/subsector system is singular where the auxiliary energies vanish simultaneously. More generally, any reachable component that, at the target point, neither provides an active slot with nonzero signed energy nor has a nonzero total exponential energy necessarily has a zero letter.

The general conclusion for $M_1 = 0$ has already been given by Eqs. (75)–(77): a general mixed state must consult the endpoint asymptotics separately; the pure-massless exception was explained after Eq. (76), where $M_1 = 0$ corresponds exactly to the divergence of the unregularized $\Gamma(\alpha_v)$ and an analytic regulator must be introduced first. The boundary and recurrence producers should

consume the regularized generic parameter system, and must not misreport an unregularized zero eigenvalue as a new master or a physical failure of the recurrences.

If the target point itself lies on $M_0 = 0$, one must first reduce at generic vertex energies and then take a controlled limit, or the user must handle that singularity separately using the output DE; a zero point must not be handed to the ordinary-point numerical transport.

The recommended flow is therefore: first choose the preferred chart according to Eq. (148), obtain the same-ordered vector from the boundary of Eq. (142), take a small but finite ordinary point inside the chart, and then let the user provide a multivariable polyline in the required homotopy class to the preferred ordinary point and the final target. "Avoid" reports a direct crossing of a letter hypersurface, "Automatic" classifies an explicitly supplied singular user point and resumes from an outgoing ordinary match on the same straight continuation branch, and "SingularityJump" explicitly selects the local crossing branch and requires confirmation of its multivaluedness. Auxiliary vertex energies may be taken to zero one at a time; a finite singular point returns either a finite value or **Infinity**. A singularity at a complex-affine turn still fails closed because the input does not uniquely specify a continuation branch.

12.6 The [2] infinity boundary and FlintNDE transport

The production entry point does not compute the defining integral directly at a finite Euclidean ordinary point. `MSBoundaryData` uses the $K_v \rightarrow \infty$ data of [2] as the only boundary source. If a vertex has no physical external leg, the program internally introduces a private auxiliary external-leg energy $k_{0,v}^{\text{aux}}$ in the same analytic dlog system, first constructs the boundary data from the corresponding damped limit $K_v^{\text{aux}} \rightarrow \infty$, and then uses FlintNDE to transport along that auxiliary coordinate to the physical target $k_{0,v}^{\text{aux}} = 0$. The user does not enter this private coordinate in `pointSequence` or `ParameterRules`. Every closed tree/time-only context, including the single-vertex massive-external family, uniformly consumes the sector DAG, components, slot registry, normalization, state order and strict rank and follows the “boundary leading term plus FlintNDE Frobenius recurrence” route. The explicit multivariable series of Eqs. (3.44)–(3.46) of [2] is retained only as a test oracle and is not a production dispatch.

Concretely, for a component C of a sector s define (every ν in this subsection is the parameter of this document after Eq. (4))

$$\begin{aligned}\alpha_{sC}(\mathbf{a}) &= A_{sC} + 1 - \sum_{r \in H(s,C)} a_r(1 - 2\nu_r), \\ \ell_{sC} &= V - \min_{v \in C} \text{rank}(v) + 1, \quad K_{sC}(t) = c_{sC} t^{-\ell_{sC}} + O(t^{-\ell_{sC}+1}).\end{aligned}\tag{159}$$

Here $H(s,C)$ contains only the still-active Hankel endpoint slots on that component; massless shared slots do not enter α_{sC} , but instead give a ± 1 state factor according to the fixed theta chamber. The Frobenius exponent and the physical leading weight of the corresponding master are

$$\begin{aligned}\lambda_{s,\mathbf{a}} &= \sum_C \ell_{sC} \alpha_{sC}(\mathbf{a}), \\ W_{s,\mathbf{a}} &= \mathcal{N}_s \Xi_{s,\mathbf{a}} \prod_C [(-i)^{A_{sC}+1} \Gamma(\alpha_{sC}) E_{sC,\mathbf{a}} c_{sC}^{-\alpha_{sC}}].\end{aligned}\tag{160}$$

$\mathcal{N}_s$ is the sector normalization, $\Xi_{s,\mathbf{a}}$ collects the shared-massless chamber sign, and $E_{sC,\mathbf{a}}$ is the product of all Hankel endpoint coefficients of the component. Each contracted massive Full line e contributes the pinch factor of Eq. (2.17) of [2],

$$-\frac{4i}{\pi} \eta_e k_e^{2\nu_e-1}, \quad \eta_e = \begin{cases} +1, & ++ \\ -1, & -- \end{cases},\tag{161}$$

to $\mathcal{N}_s$. The factor η_e defines only the normalization of the physical child master; it is not multiplied again into the normalized-master dlog contact block or recurrence event. The remaining constant coefficient of a massive contact is already complete in the defining-integral coefficient of each component, and no extra phase may be multiplied according to contact depth. All quantities are taken from the current sector metadata; one must not guess the child powers or two-state factors from the root sector.

Pull back the complete dlog potential along $K_{\sigma(j)} = B^{V-j+1}t^{-(V-j+1)}$ and denote the residue by R_0 . For each $(s, \mathbf{a})$, the program fixes the component of the corresponding root master in that sector to 1 and the components of the non-ancestor sectors to 0, and solves

$$(\lambda_{s,\mathbf{a}}I - R_0)v_{s,\mathbf{a}} = 0. \quad (162)$$

Then $W_{s,\mathbf{a}}v_{s,\mathbf{a}}$ automatically contains the leading components injected into the parent by the child solutions. Repeated roots or integer-difference resonances do not create graph-specific parameters; the complete exact residue together with these $\{\lambda, 0, v\}$ is handed to the power-log recurrences of FlintNDE. Finite defining integrals are independent-validation oracles only and never production inputs.

Eq. (4.2) of [2] gives the remaining term of the two-vertex case with equal time powers ν_0 . It is written below in the $+\nu$ convention of this document via Eq. (4); the superscript paper only records that the endpoint basis of that paper is still in use, with conjugate Hankel orders at the two ends, and does not denote another prefactor convention:

$$I_R^{\text{paper}} = -\frac{4i}{\pi}e^{-\pi \text{Im } \nu_1}k_s^{2\nu_1-1} \int_{-\infty}^0 d\tau (-\tau)^{2\nu_1-2\nu_0} e^{i(k_{12}+k_{34})\tau}. \quad (163)$$

This expression cannot be copied directly into the MadStree sector normalization without a basis conversion. For a heavy physical order, whose Hankel orders are $\pm i\mu$ with μ real, the conjugate-order endpoint of [2] and the common-order endpoints of MadStree obey

$$e^{-\pi\mu}H_{-i\mu}^{(2)}(z) = H_{i\mu}^{(2)}(z). \quad (164)$$

The explicit Boltzmann factor in Eq. (163) is therefore absorbed by the endpoint-basis conversion. This example is G_{++} , so Eq. (161) has $\eta_e = +1$ and the child normalization in the common-Hankel-order MadStree basis is

$$\mathcal{N}_R^{\text{MS}} = -\frac{4i}{\pi}k_s^{2\nu_1-1}, \quad (165)$$

and must not be multiplied by $e^{-\pi \text{Im } \nu_1}$ again. The same fifth branch has the leading vector and the physical coefficients of the two endpoint bases

$$\begin{aligned} v_5 &= \left(0, \frac{1}{\nu_0 - 2\nu_1}, \frac{1}{1 - \nu_0}, 0, 1\right)^T, \\ C_5^{\text{paper}} &= -\frac{4i}{\pi}e^{-\pi \text{Im } \nu_1}k_s^{2\nu_1-1}e^{-i\pi(2\nu_1-2\nu_0+1)/2}\Gamma(2\nu_1 - 2\nu_0 + 1), \\ C_5^{\text{MS}} &= -\frac{4i}{\pi}k_s^{2\nu_1-1}e^{-i\pi(2\nu_1-2\nu_0+1)/2}\Gamma(2\nu_1 - 2\nu_0 + 1). \end{aligned} \quad (166)$$

This uses the $i\epsilon$ branch $\int_{-\infty}^0 d\tau (-\tau)^p e^{iK\tau} = (-i)^{p+1}\Gamma(p+1)K^{-p-1}$. The phase of Eq. (4.11) as printed in [2] reads $e^{i\pi(\nu_0-\nu_1)}$ in the convention of this document and is larger by one factor of i than this direct evaluation of the defining Eq. (163). MadStree follows its public definition $J_s = \mathcal{N}_s I_s$, uses the direct integral, and explicitly maps to the same endpoint basis before comparison with [2]. The base formula stored by the code is exactly this $+\nu$ version of the present document (the order slot reads `formulaNu =  $s_\nu\nu$` , by default $+\nu$); to compare verbatim with the printed formulas of [2] one only replaces the order by that reference's $\nu_{\text{ref}} = -\nu$, and none of the formulas above requires a further replacement. The five-dimensional result above is now merely a check example

of Eqs. (159)–(162), not a production dispatch; general unequal time powers read the A_{sC} of each component directly. The massive pinch normalization for real positive physical momenta is fixed by Eq. (165); the formal symbol $(-k_s)^{2\nu_1-1}$ must not be handed to the Mathematica principal branch.

MSEvaluatePath partitions the input order into maximal consecutive complex-affine one-variable segments $x(s) = x_0 + sv$ and pulls each segment back only once. It sends every exact complex parameter point, the boundary, and the selected options in one UTF-8 request. **MadStree** neither selects nodes nor builds detours; **FlintNDE** initializes the boundary, optionally plans, and transports in one process. With planning enabled, points covered by one expansion node form an evaluation bucket: large buckets use subproduct/remainder-tree fast multipoint evaluation and small buckets use the iterative algorithm. With planning disabled, every user point is a transport node. The second argument is always named **pointSequence**. Its first row is the ordered header of running-coordinate symbols and every later row is an equal-width value row. A single point has one value row; multiple points only append rows. Fixed, non-running parameters are supplied once through **ParameterRules**. The header and fixed parameters must be disjoint and together cover every symbol required by the analytic DE and boundary. Missing or still nonnumeric values return a Failure listing the symbols before a runtime directory is created or Python/FlintNDE starts. The function, options, backend schema, **pointResults**, and return structure do not depend on the number of points. **TransportOrder** sets the production primary chain; nodes, dense points, segment endpoints, and regulator-fit samples all come from this lower-order chain. The higher **ReferenceTransportOrder** chain is used only for the relative-difference accuracy check. Regular-singular initialization and successive affine segments propagate both states independently, so a reference endpoint neither replaces user output nor restarts the next primary chain. Distinct segments never share local coefficients. A single project-relative configuration variable controls the backend location. **MSRuntimeDirectory** denotes the temporary runtime root itself; **Automatic** creates one **results_temp/** next to the calling script, with short **nde/** and **cache/** children, never in the package source tree. On Windows, complete cache, input, and log paths are checked before directory creation or Python launch. Paths longer than 259 characters return the separate **RuntimePathTooLong** failure. Input writes, missing **python-flint**, launch failures, missing output, and invalid output retain distinct tags. CSV/JSON export returns **"written"** only when every requested format has been written successfully.

The analytic **MSDLogDE[context]** is itself a durable user result. A symbolic-only workflow may call **MSWriteFormulaArtifacts** explicitly. **MSEvaluatePath** and **MSReconstructEpSeries** automatically write or reuse the complete formal artifacts before any Python/FlintNDE task starts. A write failure prevents numerical NDE, and the successful result reports every path under **"formulaArtifacts"**. The default files are **masters.wl**, **recurrence_metadata.wl**, **dlog_de.wl**, and **manifest.wl** under **results/madstree_formula/run-<UUID>/** by default. The **dlog** file always contains the complete analytic DE. Numerical work derives only a function-local parameterized copy and never overwrites the analytic object or durable file. Repeated evaluations in one session reuse the complete files for the same context and calling root; deleted files are regenerated, while any partial write fails before backend launch. Ordinary-point export defaults to **results/madstree_evaluation/run-<UUID>/evaluation_data.csv/json**. **results/** contains durable user outputs, **results_temp/** contains requests, logs, and cache, and **test/results_test/** contains development-test artifacts.

Every multipoint transport through intermediate nodes is a jump; only a crossing of a singularity through a local-basis connection is a singularity jump. The default **SingularityMode -> "Automatic"** follows explicitly supplied singular user points and warns about the local branch. An exact singularity is removed from the ordinary-node chain; one local basis owns user points on both sides inside the disk whose radius is the distance to the nearest other singularity. A removable singularity returns a numerical value, while a true pole or a zeroth-order logarithmic divergence returns the text **Infinity**; transport then resumes from the outgoing ordinary match. A terminal singular target receives a hidden in-disk ordinary match point along the

incoming direction when required. The lower-order primary chain supplies the result, and the higher-order reference chain only checks classifications and finite values. The evidence is listed separately in `singularityClassifications`. "Avoid" explicitly refuses a crossing, whereas "SingularityJump" explicitly selects the local crossing branch and warns the user to confirm the multivalued branch of the equivalent detour class. A non-collinear singular turn, consecutive singular points, and an intermediate singular point with planning disabled remain fail closed. `MessageLanguage` $\rightarrow$ "EN" gives English notices by default, while "CN" selects Chinese; the values are case-sensitive. The backend discovers finite singularities from each pulled-back one-variable system, so the user does not supply a list.

If every dlog letter is constant on a segment, every pulled-back derivative vanishes, so the segment is a zero connection with no finite poles and is transported normally. Both `MSBoundaryData` and `MSEvaluatePath` default to 200 decimal working digits; automatic regulator planning takes the larger of 200 and its adaptive estimate, while an explicit precision overrides the default. Flint-NDE uses $\lceil p_{\text{dec}} \log_2 10 \rceil + 32$ working bits: the 200-digit default gives 697 bits, while 70 and 100 digits use 265 and 365 bits, and higher requests continue to grow by the same formula; the numerical gates of the local singular basis tighten with the current precision as well. Segment projection, match ratios, rotation factors, and winding/monodromy retain the current Arb precision. All geometry retains the current Arb precision.

12.7 Symbolic Laurent-support certificate and regulator error control

Let the shared regulator be ϵ . In `MSReconstructEpSeries[... , MaximumEpPower  $\rightarrow$   $m$ ]` the user specifies only the highest returned power m ; the lowest power $n_{\min}$ is not a numerical fit parameter. Before starting any NDE task at nonzero ϵ , MadStree constructs a support certificate from the physical boundary and dlog DE of the same production context. It requires:

1. regulator-independent path coordinates and no negative Laurent power at $\epsilon = 0$ in any dlog matrix, pulled-back connection, or residue;
2. finite-Laurent defining-integral boundary branches; branches with the same limiting Frobenius exponent and log degree are first combined with their physical weights and leading vectors, and only then are component valuations extracted;
3. at least one provably nonzero coefficient at the lowest power of the combined boundary. A noninteger power, non-Laurent structure, undecidable coefficient, or support entirely above the checked user range fails closed.

Before these symbolic valuations, the package also checks that the numerical input is complete and that a generic boundary chart can be constructed. This coverage preflight uses the fixed nonzero exact probe $\epsilon = 1/1009$, so a legitimate normalization pole is not rejected by a generic numeric-completeness check. The probe is neither a production nor a validation sample, never enters the fit, and does not determine $n_{\min}$; the dlog DE, complete physical boundary, and resonance records are still Laurent-valued strictly at $\epsilon = 0$. For $n_{\min} < 0$ the certificate describes a finite meromorphic Laurent boundary, not an analytic boundary at $\epsilon = 0$; analyticity is recorded only when $n_{\min} \geq 0$. Along the current finite path, an ϵ -analytic invertible fundamental matrix preserves the valuation of nonzero formal boundary data, so the certified $n_{\min}$ is passed directly to numerical sample planning. A rank drop of an individual Frobenius recurrence operator at $\epsilon = 0$ is retained as a resonance diagnostic only; it is not by itself interpreted as a $1/\epsilon$ pole across the already combined physical boundary.

Example 06 uses the massless three-vertex family with $a_1 = a_2 = a_3 = 1 + \epsilon$. Its physical boundary weights include $\Gamma(2 + \epsilon)^3$, $\Gamma(2 + \epsilon)\Gamma(4 + 2\epsilon)$, and $\Gamma(6 + 3\epsilon)$, all regular at $\epsilon = 0$. The actual nine-dimensional DE also has no negative ϵ power. The program therefore certifies $n_{\min} = 0$ before any NDE solve: this example has no pole, and no artificial $1/\epsilon$ term is introduced.

After support certification, the square interpolation initially includes at least two powers above the user maximum. Independent validation points never enter the fit. If their relative residual exceeds the fixed `EpGoalDigits` target, the next round adds two fitted powers by default, solves only the new production points, and reuses every previous production and validation value. If the target is still missed after at most three rounds, the current best coefficients are returned without precision certification: the tolerance is not relaxed, least squares is not substituted, and caches are not discarded. Independent regulator tasks use an outer process pool with default `ParallelTaskCount` $\rightarrow$ 12; the effective count is the smaller of this limit and the number of tasks.

The default call still contains no regulator values. To restrict every solve to an approved range, `EpSamplePoints` $\rightarrow$ `{...}` supplies an ordered production candidate pool and `EpValidationPoints` $\rightarrow$ `{...}` supplies an independent grid disjoint from that whole pool. The pool may be redundant. With `EpInitialInternalMaximumPower` $\rightarrow$ q , the first round consumes exactly the $q - n_{\min} + 1$ candidates required for powers $n_{\min}, \dots, q$; later candidates are consumed only after failed validation. Exhaustion never creates an out-of-pool point: the current coefficients are returned with `computed_with_warning`, `precisionTargetMet` $\rightarrow$ `False`, and reason `candidate_pool_exhausted`.

Alternatively, `EpSampleAngleRange` $\rightarrow$ `{thetaMin, thetaMax}` selects an open angular interval in radians. Up to three uniformly spaced interior rays are used, while magnitudes remain controlled by the same precision formula and have no user radius cap. Validation points retain their corresponding angles and reduce only the radii. The high-precision real and imaginary parts are rationalized to exact values in $\mathbb{Q}(i)$ before `MadStree` performs its exact dlog pullback. The option is mutually exclusive with explicit `EpSamplePoints`; `Automatic` keeps the original positive-real-axis schema. All related options default to `Automatic`, preserving the original default call and adapter JSON field set.

Ordinary coordinate-value rows are saved; a transient waypoint is written as `{{value1,value2,...},"tmp"}`. Ordinary saved points in the `MSEvaluatePath` result carry coordinates, the full master-integral vector, status, and user index. Use `MSEvaluatePath[result, MSOutputDirectory  $\rightarrow$  outputDirectory, ExportFormats  $\rightarrow$  {"CSV", "JSON"}]` to export them. The option accepts `Automatic`, either single format, or a list of the two. Status is `written` only when every requested format succeeds. Precision and relative-difference fields that are already strings round-trip through JSON without an additional layer of literal quotes. Transient waypoints are excluded from the ordinary-point export.

`FlintNDE` classifies the local singularities and runs a capability preflight before starting; the current generic boundary producer of `MadStree` delivers $\{a, b, C\}$ only when the pulled-back connection is an exact regular-singular one, so the addition of $\{\phi, a, b, C\}$ is a backend path-saving capability rather than a change of the dS boundary source. If the input requires higher-order or irregular singularities beyond the declared capabilities of the backend, the adapter fails closed directly and does not disguise ordinary Taylor transport as a singular solution.

In the original-coordinate formulation, the $e^{-K_v t_v}$ damping of the integrand looks exponentially asymptotic at $K_v \rightarrow \infty$; this does not mean that the dlog system of the normalized master integrals necessarily has an essential singularity at the blow-up coordinates $x_j = 0$. After extracting the known physical weights of Eq. (160) and performing the nested blow-up, the currently accepted exact systems must be ordinary matching problems or Fuchsian regular-singular problems. If the actually pulled-back system still has an irreducible pole of order higher than two, needs Stokes/ramification, or is not in the current exact field, the capability preflight returns the actual classification and stops; the package does not automatically interpret it as a solved boundary condition.

The following are already checked end to end: the [2] boundary for a single vertex with three massive lines, the analytic chamber boundary for the two-vertex pure-massless case, the generic 15-dimensional boundary and transport for the three-vertex massive/massless case, and

the five-dimensional G^{++} two-vertex system. For the pure-massless defining integral, both DE residuals of the contact-atom weights and the complete target vector are checked; for the massive five-dimensional system, the connection, the singular pullback, the five leading vectors, the five coefficients and the target vector all agree with an independent paper-based route. Structural checks also cover common-theta simultaneous events, massless/massive coincident quotients, and the generic boundary/residue of the triangle time-only loop graph. The FlintNDE adapter additionally checks ordinary/regular-singular save points and the fail-closed behavior for super-capability poles. Representative run evidence is not a point-by-point certification of all topologies and parameter regions; the generality comes from the same metadata data flow and the joint regression over structurally different topologies, not from an enumeration of dedicated production branches.

13 Typical usage examples

13.1 Single-vertex family: definition, reduction, and unified point input

Use `MSInitVertexFamily` for a function product at one vertex instead of constructing a fake one-vertex topology. Package example `Examples/02_single_vertex_family.wl` gives both the compact and equivalent explicit models, reads the masters and dlog DE, and reduces one shifted integral. Its numerical section uses only one entry:

```
parameterRules = {q->1,nu->1/2,a->2};
singlePointSequence = {{k0},{3 I}};
multipointSequence = {{k0},{3 I},{7 I/2},{4 I}};
single = MSEvaluatePath[ctx,singlePointSequence,
  ParameterRules->parameterRules];
multi = MSEvaluatePath[ctx,multipointSequence,
  ParameterRules->parameterRules];
```

The example compares the common first point component by component, demonstrating that a single point is merely a length-one `pointSequence`.

13.2 Two-vertex massless Full edge: multiple sectors and points

`Examples/01_massless_full_edge.wl` starts from two + vertices and one public "massless" line. `MadStree` derives the Full/quotient classification, top and child sectors, masters, contact map, and dlog DE. The example uses `{k1,k2}` as the header, puts only coordinate values in later rows, and supplies common `q/a1/a2` values once through `ParameterRules`. It then enables `FlintNDE` planning and fast multipoint evaluation and exports saved points with `MSEvaluateEvaluationData`. A transient waypoint is `{{value1,value2},"tmp"}`; there is no separate path object.

13.3 Massive three-vertex tree: zero middle external-leg energy is not automatically singular

`Examples/05_massive_three_vertex_tree.wl` initializes one context for a three-vertex tree with two massive lines. It fixes $k_3 = 5i$, $q_{12} = 1$, $q_{23} = 2$, $\nu_{12} = 3/4$, $\nu_{23} = 1/3$, and $a_1 = a_2 = a_3 = 1$. The original sequence

```
{{k1,k2},{8 I,2 I},{7 I,4 I},{6 I,3 I},{9 I,5 I}}
```

is retained. A second independent `MSEvaluatePath` call plans, evaluates, and exports

```
{{k1,k2},{8 I,0},{7 I,0},{6 I,0},{9 I,0}},
```

so the two runs share only the context and never mix their plans or results.

The singularity test substitutes each point into the complete top/contact-sector dlog letters; it does not test whether an individual coordinate is zero. No letter vanishes at these four points, so all are ordinary: all 25 master-integral components are saved, the `singularityClassifications` table is empty, and the maximum pointwise relative difference between the lower-order primary chain and the higher-order reference chain is about 2.31×10^{-31} . If another family really meets an exact letter at zero or at any nonzero complex coordinate, the same FlintNDE local power-log entry classifies it as removable, pole, or log divergent without dispatching on a variable name or graph type.

13.4 Three-vertex regulator: automatic Laurent support and incremental fitting

`Examples/06_massless_three_vertex_ep_regularization.wl` sets $a_1 = a_2 = a_3 = 1 + \epsilon$, while the user requests only `MaximumEpPower->0`. The package first certifies the lowest power from the symbolic boundary and DE and then selects production and independent validation points. If validation misses the target, only the internal fit order and new points are added, while previous values are reused. Its coordinate table contains only ϵ -independent k_i values, while $a_i \rightarrow 1 + \epsilon$ occurs only in `ParameterRules`. This example certifies lowest power zero and therefore returns the finite part without inventing a pole.

13.5 No physical external leg: private auxiliary energy to zero

`Examples/07_zero_external_leg_energy.wl` shows the two equivalent inputs for a vertex without a physical external leg: omit `externalLegEnergy` or set it explicitly to zero. The example puts only physical coordinates in `pointSequence`, verifies that the private auxiliary coordinate remains absent from public input, and transports it from the damping boundary to zero while preserving the same analytic dlog system.

14 Minimal algorithm and acceptance of the direct formula producer

Without running a sampled IBP or reduction, the formula producer only needs to perform the following steps:

1. Build the full-graph building-block registry from the topology: a two-dimensional h slot for each massive endpoint and one shared two-dimensional slot for each uncontracted massless full edge; also store the slot incidence list of each vertex. Theta-free exponentials do not enter the registry and only give the scalar p_0 of each component through Eq. (58).
2. Fix S_e, P_e of Eq. (30) and the endpoint incidence for each massless full edge; cross/external exponents do not create any slot.
3. Assemble the Kronecker sums vertex by vertex according to Eqs. (45)–(46); the one-dimensional exponential terms and all two-dimensional terms use the same embedding API, and ς_v, σ_e and ϵ_{ve} are kept strictly separate.
4. First verify Eq. (49), then enumerate the diagonal elements of Eq. (52) with one global U_s ; invert only the scalar diagonal elements of M_1 and $\widetilde{M}_0$.
5. Run an event BFS over the current-component bundles according to Eq. (82); construct the S_s, P_s quotient for coincident spectators, and fix the child powers with Eq. (83).

6. Output the general-index one-step recurrences with Eqs. (68)–(71); append the lower-sector sources with the event-level contact selector, and return the M_1/M_0 singular layers by direction.
7. Output the block-triangular dlog candidate directly with Eqs. (87) and (90); nonzero child shifts are reduced back to the global masters automatically by Eq. (99), and then Eq. (132) is applied.
8. Call `MSBoundaryChartCertificate` for the chosen or all root ranks; only when the complete connection, normalization, Jacobian, theta fixing and quotient all pass is the automatic boundary entered.

The minimum exact acceptance should include:

- the $4 \rightarrow 2$ rank of a single massless full edge, $P_e S_e = I_2$, Eqs. (32)–(33), and the contact vector;
- in a two-vertex single-edge graph, the same shared bit has opposite signs in the two κ_v ;
- the four common eigenstates of the two shared slots in a three-vertex chain, rather than an independent enumeration per vertex;
- the global $U_s M_{0,v,s} U_s^{-1}$ is fully diagonal for a mixed massive/massless tree;
- the formula one-step relations equal the naive time-IBP term by term in the exactly same ordered normalized basis;
- the formula DE equals the naive DE matrix element by matrix element, all lower-sector residuals are empty, and exact flatness is passed;
- in the mixed three-vertex case, the five 15×15 matrices obtained from the public `ds` of `dSIBP` through the slot-by-slot adapter and `MSReduce` are strictly equal to the direct formula DE;
- the physical output basis additionally contains Eq. (109), without missing k , k^2 or the normalization derivative.

15 Final assessment

The purely massive formulas are Kronecker sums on the building-block registry from the outset: the theta-free exponential of each vertex gives $\Delta M_1 = 0$ and $\Delta M_0 = ip_{0,v} I$, each two-dimensional h endpoint gives $\Delta M_1 = -(1 - 2\nu_i) P_1$ and $\Delta M_0 = -ik_i \sigma_2$, and all terms are multiplied by the identity matrices of the other slots; every ν_i here is the parameter of this document after Eq. (4), and the default positive-prefactor convention of `MadStree` requires no further replacement. Positive and negative vertices only change the sign ς_v of the pure vertex exponential; along $k_{0,v} = -i\varsigma_v K_v$ with $K_v > 0$, both become $e^{-K_v t_v}$ and the two-dimensional h atoms are unchanged.

After a massless full edge containing theta is included, one should first keep the same two endpoint two-states as for a massive edge, and then reduce the four states to one shared two-state with S_e, P_e . The two vertices no longer have mutually independent factors of that edge; instead they act on the same slot with opposite signs. The vertex-by-vertex tensor factorization therefore fails, but the full-graph Kronecker sum, the common diagonalization, the iteration matrices and the dlog DE can still all be written directly. The theta derivatives only add the lower-sector block generated by $|1\rangle_e$. The $F_{11} = +F_{00}$ of the reference h-state and the $F_{11}^J = -F_{00}^J$ of the package quotient are strictly connected by the constant D_{σ_e} and must not be mixed.

The single-vertex recurrences are not blocked by the shared slot: adding the blocks of the vertex's incidence list gives $M_{1,v,s}$ and $M_{0,v,s}$. Different vertices merely share the state bit on

the same massless slot and take opposite endpoint signs. At present, only one fixed Pauli direction appears in each two-dimensional slot, so operators on the same slot commute pairwise; the Kronecker product of the local 2×2 diagonalizers reduces the large-matrix inversion to a letter-by-letter reciprocal of the energy letters. If non-collinear Pauli directions appear on the same slot in the future, this fast path must be disabled. Lowering fails only when an M_1 eigenvalue is zero, and raising fails only when an energy letter of M_0 is zero; a matrix that is not inverted in that direction produces no pole in that direction even if it is zero.

The dlog DE is produced directly by the same registry: the sector diagonal blocks use the commonly diagonalized $\log \kappa$ with $M_1(A+1)$, the top-to-sub blocks use the same $\log \kappa$ times the $R^{(1)}$ contact map, and then the sector normalization is applied. Hence there is no need to generate a large IBP system first and then solve for the DE; but only after the normalization ratio, the constant residue, the block closure and the flatness all pass can the output be certified as a dlog, rather than merely called a connection candidate.

The boundary likewise uses the initialized $K_v = i\varsigma_v k_{0,v}$ and does not permanently rename the user variables to `pik0/mik0`. The preferred ordinary point selects the preferred strict chart by descending $\text{Re } K_v$; this is equivalent to ascending $|\tau_v|$, because a large damping energy supports only a smaller typical $|\tau_v|$. This choice serves to shorten the first continuation segment and reduce the letters that may need to be bypassed; it does not replace the ordinary-point and normal-crossing certificates.

The massless building block should always be written with the dimensionless h equation $h_{zz} + h = 0$. The physical form $\partial_\tau^2 h = -k^2 h$ is fully equivalent; it is only the two chain-rule steps of $\partial_\tau = -k \partial_z$. It does not require introducing another incompatible set of local EOMs. In this way the 2×2 structure of the reference formulas is preserved, and k^2 and the contact are correctly recovered in the final physical basis.

References

- [1] Jiaqi Chen and Bo Feng. Towards Systematic Evaluation of de Sitter Correlators via Generalized Integration-By-Parts Relations. *JHEP*, 06:199, 2024. doi: 10.1007/JHEP06(2024)199.
- [2] Jiaqi Chen, Bo Feng, and Yi-Xiao Tao. Multivariate hypergeometric solutions of cosmological (dS) correlators by dlog-form differential equations. *JHEP*, 03:075, 2025. doi: 10.1007/JHEP03(2025)075.